Fundamentos de Aprendizagem Automática

Class 03 — Linear Models

Mário Antunes

Universidade de Aveiro

2026

From Optimization to Models

Where We Left Off



- Class 01: a good model **compresses** the data; we measure it on **unseen** data
- Class 02: training is $\theta^\star = \arg\min_\theta \mathcal{L}_{data}(\theta) + \lambda\,\Omega(\theta)$
- Today: three classic models, each one *a choice of* f_θ *and* $\mathcal{L}$

Three Models, One Recipe

Model	Predicts	Loss	Fit by
Linear regression	a number	squared error	closed form or GD
Perceptron	a class	$\max(0, -y\, f(x))$	online update
Logistic regression	a probability	cross-entropy	GD (no closed form)

- All three are trained with **`jax.grad`** in the lab: we write the loss, JAX gives the gradient
- Polynomial features and regularization can be added to any of them
- Every one is compared with its **scikit-learn** twin: same split, same scaling, a printed difference

Two Datasets

Spam detection (classification)

- Spambase: **4601** e-mails, **57** features
- Word and character frequencies, capital runs
- **39.4%** spam
- Most values are 0, a few huge
- 571 duplicate rows

House prices (regression)

- California housing: **20640** districts, **8** features
- Income, age, rooms, occupancy, latitude, longitude
- Target: median value (100 000 USD)
- 992 districts capped at 5.0
- Up to 1243 people per house

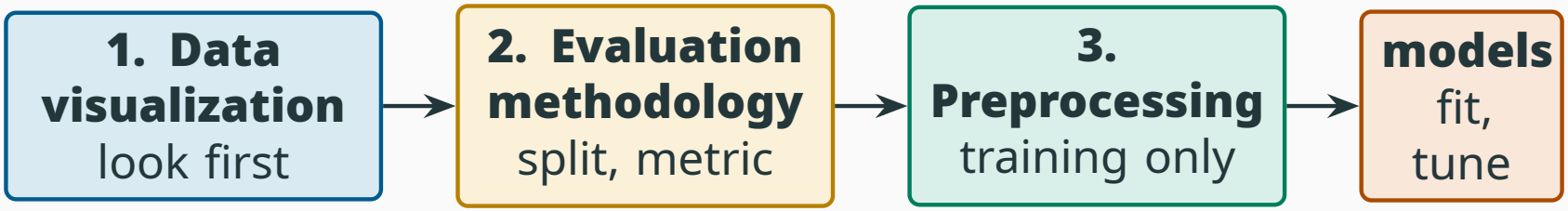
- Real, noisy, small: every demo runs in seconds (`datasets/*.csv.zst`, read with polars)

Project 1 Is Released

- **The five classes of learners:** symbolists, connectionists, evolutionaries, Bayesians, analogizers
- Pick **one dataset** (suggestions in `datasets/`, classification or regression) and **one learner per class**; justify every choice
- Today's tools are what the report is judged on: data visualization, evaluation methodology, preprocessing, the right metrics
- Submission: the code and a `README.md` with identification, models, dataset, methodology, evaluation and conclusion, **with plots and diagrams**
- Specification: `projects/project01.pdf`; due at Class 08

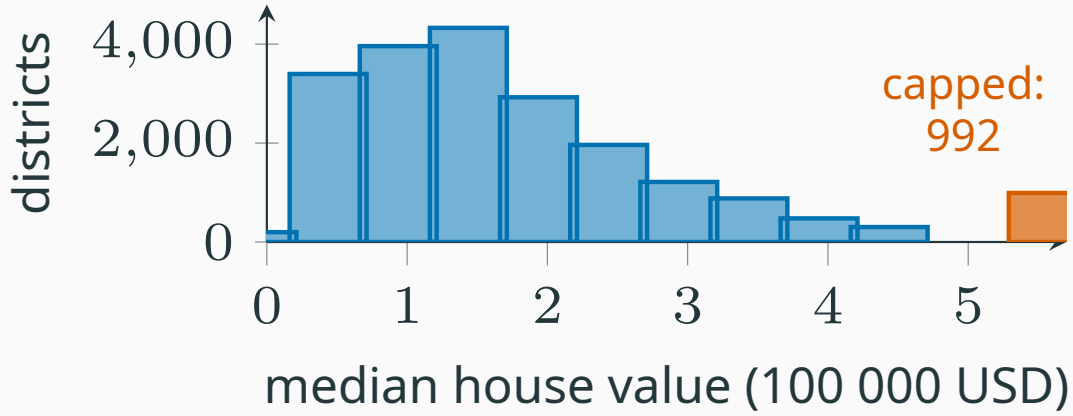
Data, Methodology and Preprocessing

Three Steps Before Any Model



- **Data visualization:** the target, the features, their relations; every plot ends in a **decision** (a logarithm, a metric, a split)
- **Evaluation methodology:** fixed *before* the results: the split, the metric that matches the cost of errors, a baseline, the spread
- **Preprocessing:** whatever is learned from data (means, vocabularies, imputed values) is fitted on the **training part only**
- The same three steps for **both** datasets, in this order

What Does the Target Look Like?



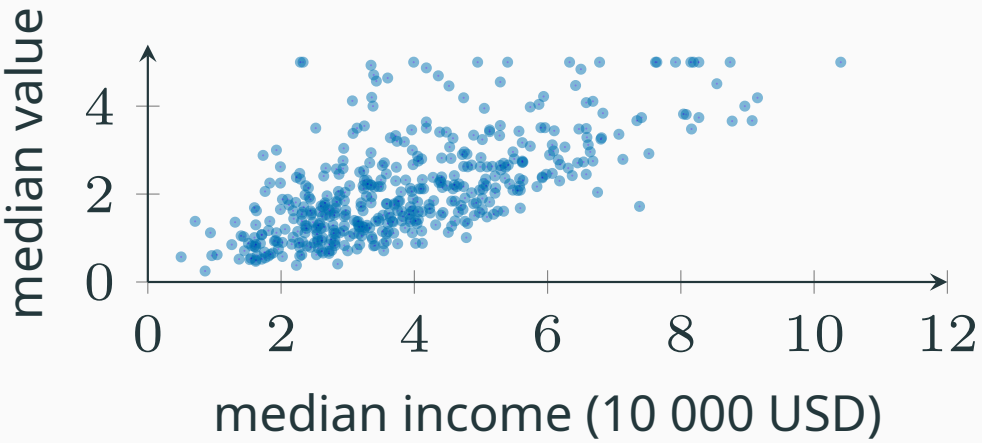
- A **histogram** counts the examples per range of values: the *shape* of what we predict
- A long tail lets a few rows dominate the squared error; a pile-up means **censored** data
- Here: right-skewed, with a **wall at 5.0** (992 districts, 4.8%): the price was capped, no model can see above it

What Does One Feature Look Like?



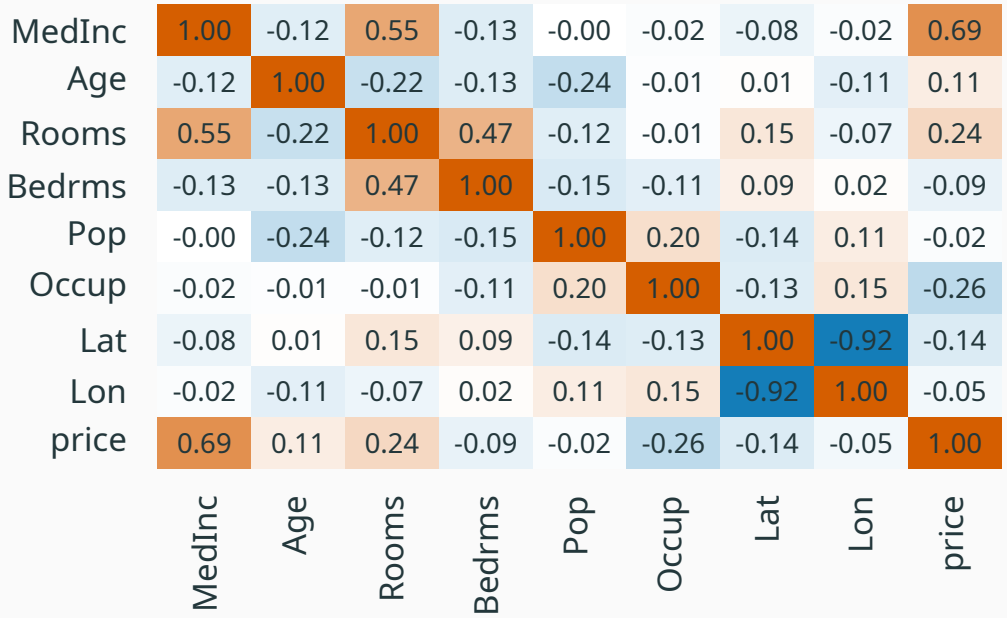
- The same histogram, now for an **input**: the average number of people per house
- It reveals scale and **outliers**: least squares, distances and penalties all react to a few extreme rows
- Here: the median is 2.8 and 92% of the districts are below 4, but the maximum is **1243**, 441 times the median

Does a Feature Predict the Target?



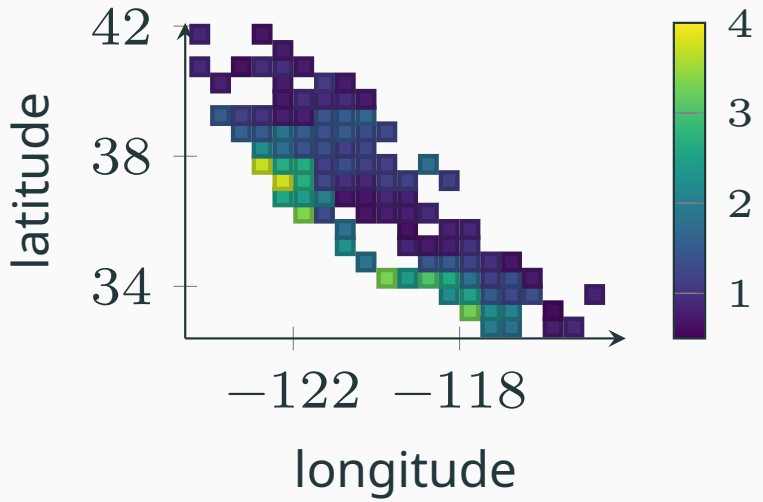
- A **scatter plot** draws one point per example: the *relation* of a feature with the target
- It tells whether a **line is plausible**, how noisy the relation is, and where it breaks
- Here (450 districts): the price rises with income ($r = 0.69$), the cloud **fans out**, and the cap shows as a flat row at 5.0

Which Features Repeat Each Other?



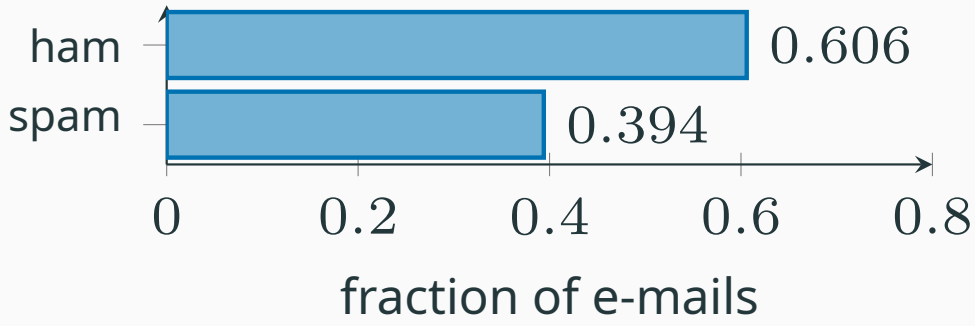
- **Pearson's** r for every pair: $+1$ together, -1 opposite, 0 no *linear* relation
- High r with the target: **signal**; high r between inputs: **collinearity**, unstable weights (regularization)
- Here: income and price 0.69; latitude and longitude -0.92; rooms and bedrooms 0.47

Does Location Matter?



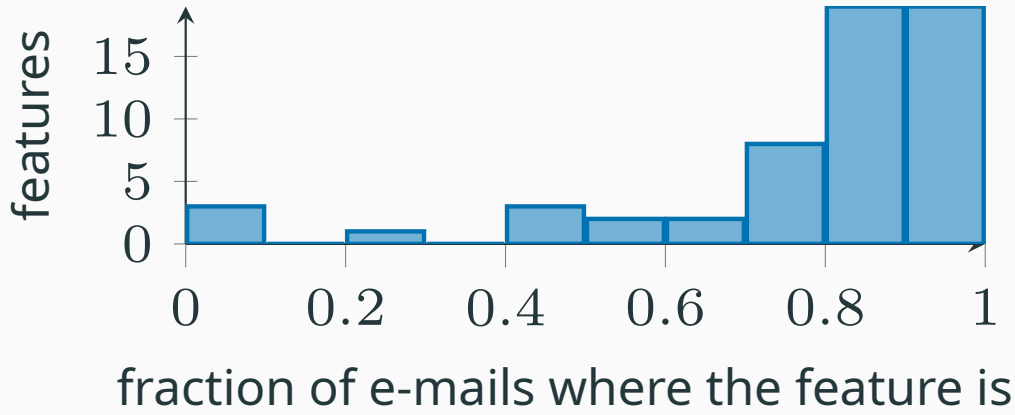
- A **map**: position on the axes, the target as colour; it exposes structure no single feature shows
- Neighbours with similar values mean a **non-linear, joint** effect: a plane cannot follow a coastline
- Here: the Bay Area and Los Angeles stand out (polynomial features, later; trees, Class 06)

How Balanced Are the Classes?



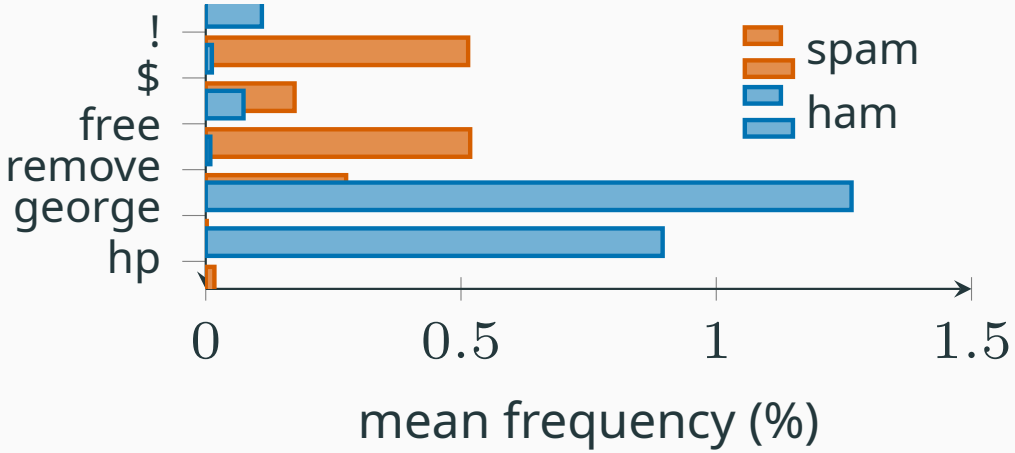
- A **bar chart of the class counts** is the first plot of any classification problem
- It sets the **baseline** (always answer the majority class), decides the use of **stratified** splits, and warns when accuracy may mislead
- Here: 39.4% spam, so “everything is ham” is already 60.6% accurate: imbalanced, but not rare

How Sparse Are the Features?



- For every feature we plot the **fraction of zeros**: how dense the data are
- Mostly-zero features mean "word absent": the mean says little, the standard deviation is set by a few rare values, and some models (Naive Bayes, Class 04) use *presence* itself
- Here: **77.4%** of the entries are 0, and 19 of the 57 features are 0 in more than 90% of the e-mails

Which Features Separate the Classes?



- Compare a feature's **mean in each class**: a large gap means the feature carries signal
- It tells us which inputs matter, and which are **artefacts** a model should not lean on
- Here: spam says `!`, `$`, `free`, `remove`; `george` and hp appear only in ham: one HP Labs mailbox, a pattern that will not transfer to other mailboxes

Do the Classes Differ in Scale?



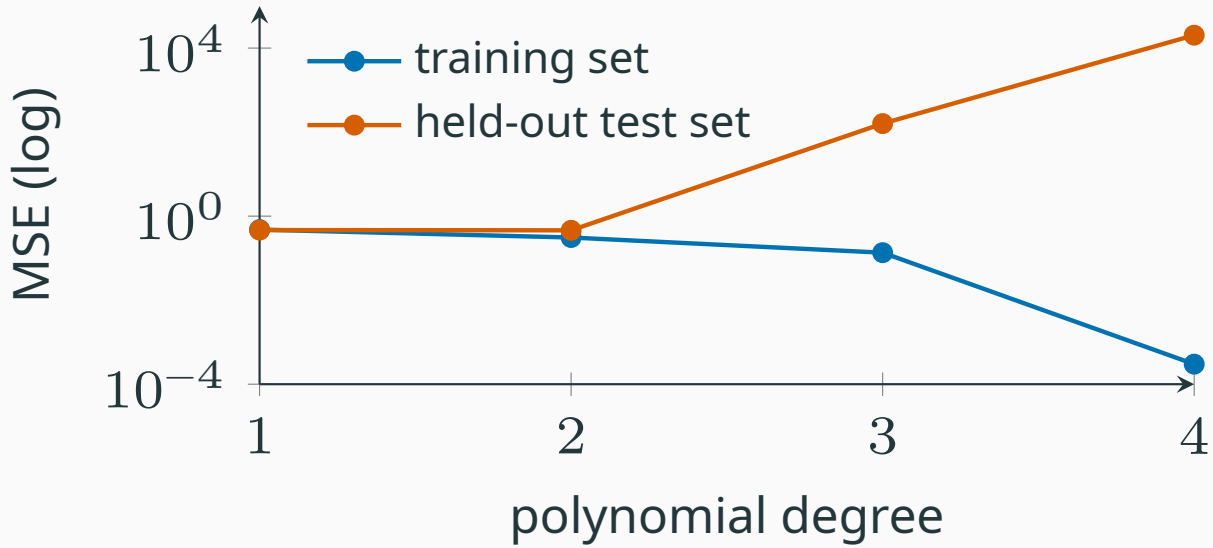
- **Overlaid histograms**, one per class, show how well a feature separates them and which scale it lives on
- A feature that spans orders of magnitude cannot be drawn, or fitted, on a linear scale: the tail hides everything else
- Here: the total of capital letters needs a $\log_{10}$ axis; spam sits about one order of magnitude to the right

From Plots to Decisions

Plot	It measures	Decision
target histogram	shape of the target, censoring	loss and metric; a floor for the error
feature histogram	scale, outliers	logarithm, scaling
scatter	form of one relation	linear or not, features to add
correlation matrix	linear association, collinearity	regularization, drop duplicates
map	joint structure	polynomial features, other models
class bars	class proportions	stratified split, MCC, baseline
zero fractions	sparsity	$\log(1 + x)$, which model
class means	signal and artefacts	features to trust

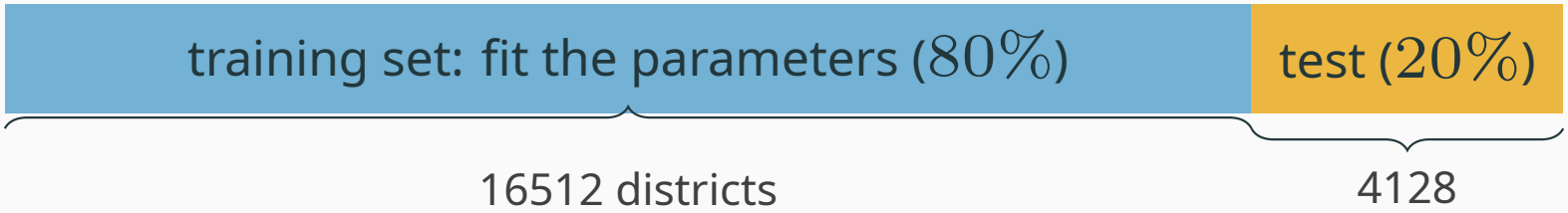
- A plot that changes no decision was not worth drawing

Methodology: Why Hold Out Data?

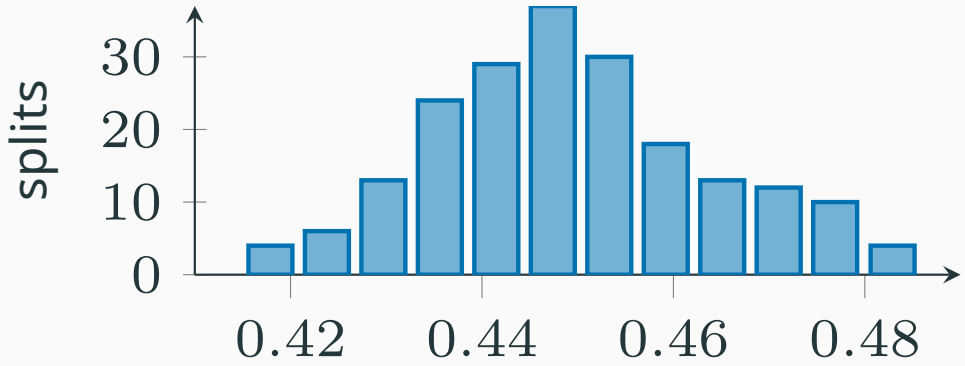


- House prices, 300 training examples, polynomial degrees 1 to 4 (demo R4)
- The **training** error says “perfect”; only **unseen** data shows the truth
- Every number we report must come from data the model did not train on

Methodology: One Hold-Out Split



- Shuffle, then cut: the model never sees the test rows while training
- Cheap, but the estimate depends on **which** rows landed in the test set



test MSE of one random 80/20 split

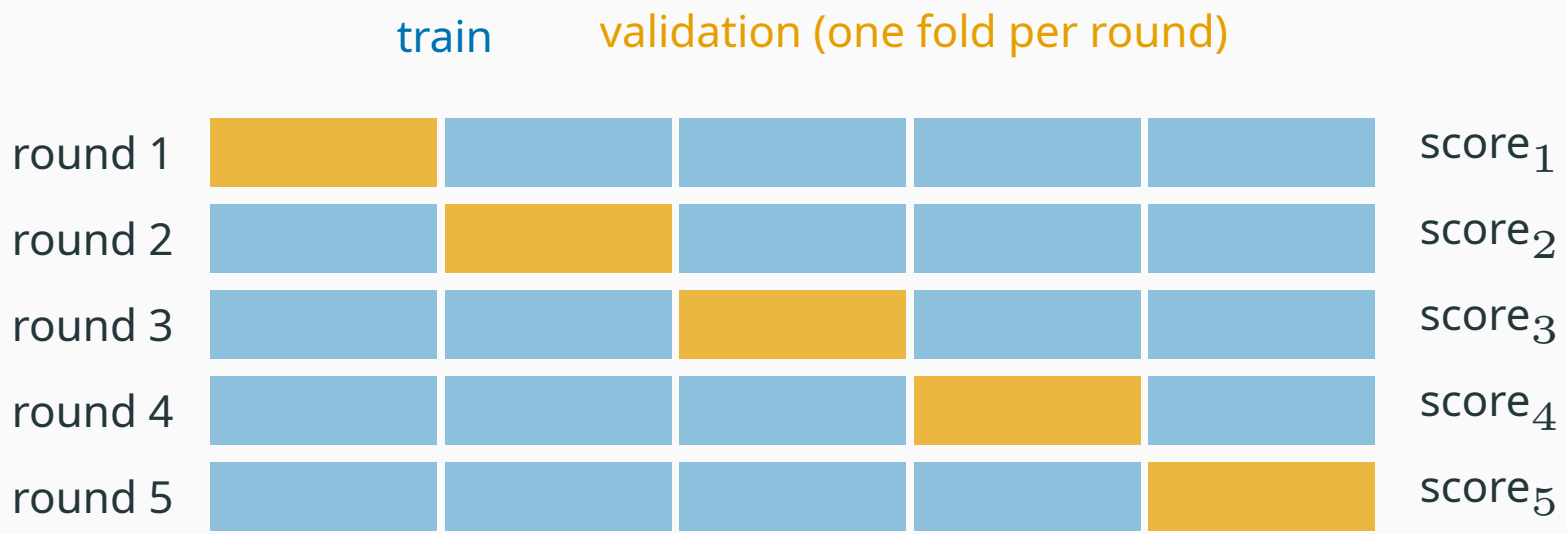
Methodology: One Split Is Not Enough

- The same linear model on **200 random splits**: test MSE from **0.415** to **0.486** (mean 0.449, std 0.015)
- With 4128 test rows the spread is small; with a few hundred rows it would be large
- **Repeated hold-out** and **cross-validation** turn one number into a mean *and* a spread

Method	Mean test MSE	Spread
one 80/20 split	0.449	± 0.015 over 200 splits
5-fold CV	0.449	std of folds 0.019, 95% CI ± 0.024
10-fold CV	0.449	std of folds 0.028, 95% CI ± 0.020

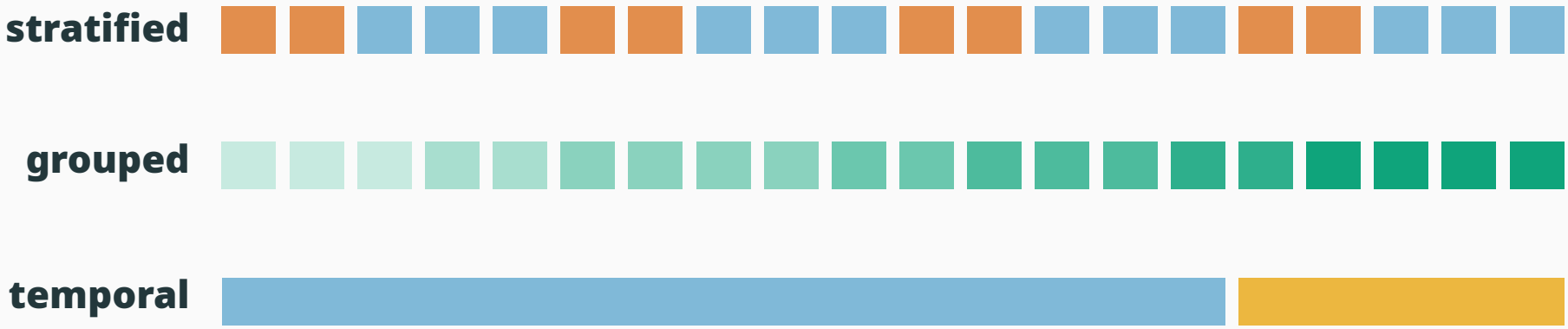
- Class 01: confidence intervals, corrected resampled t-test, McNemar

Methodology: k-Fold Cross-Validation



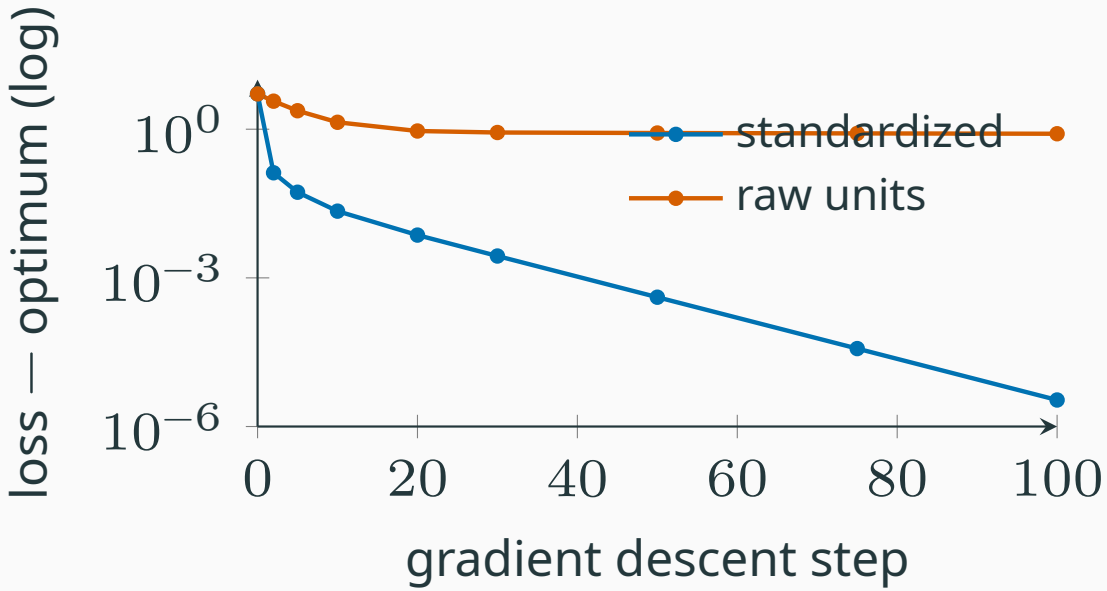
- Every example is used for validation **exactly once**; report the mean and the spread of the k scores
- Use it to **choose** hyper-parameters (degree, λ, threshold): a separate **test** set is touched once, at the end

Methodology: Stratified, Grouped, Temporal



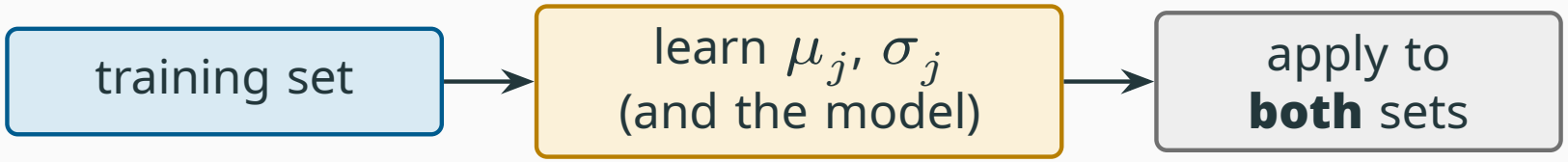
- **Stratified:** keeps the class proportions; on 100 e-mails the spam fraction of train and test differs by **0.092** on average with a random split, **0.015** stratified (lab B1)
- **Grouped:** correlated rows must stay together, or the test set leaks
- **Temporal:** never test on data older than the training data

Preprocessing: Why Scaling Matters



- Inputs in different units (latitude near 35, income near 4) make the loss a **long thin valley**: condition number **41** standardized, **3 · 10⁸** raw (Class 02)
- Gradient descent takes a tiny step to survive the steep direction and crawls along the flat one: after 100 steps the raw fit is still **0.81** above the optimum
- Penalties ($\lambda\|w\|^2$), distances (k-NN) and kernels treat all inputs alike: they need **comparable scales**

Preprocessing: Scale on Training Only

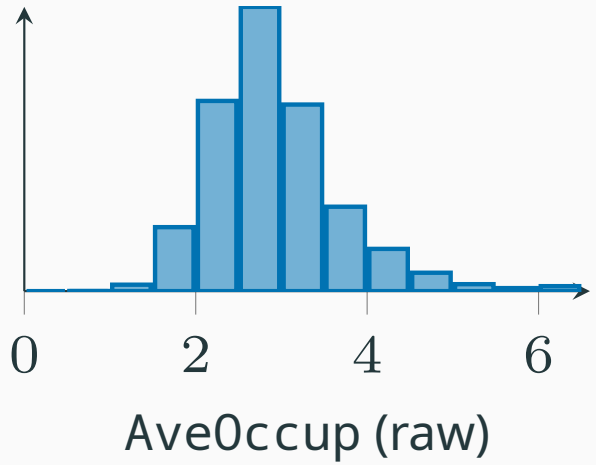


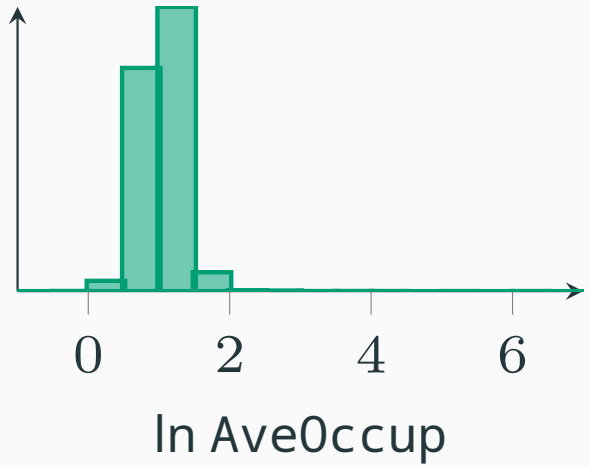
wrong: μ_j, σ_j from all the data
$\Rightarrow$ the test set shapes the model

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \qquad \mu_j, \sigma_j \text{ estimated on the training rows only}$$

- Any statistic learned from data (mean, vocabulary, bins, threshold) stays **inside** the training part (data leakage, Class 01)

Preprocessing: Why a Logarithm Matters





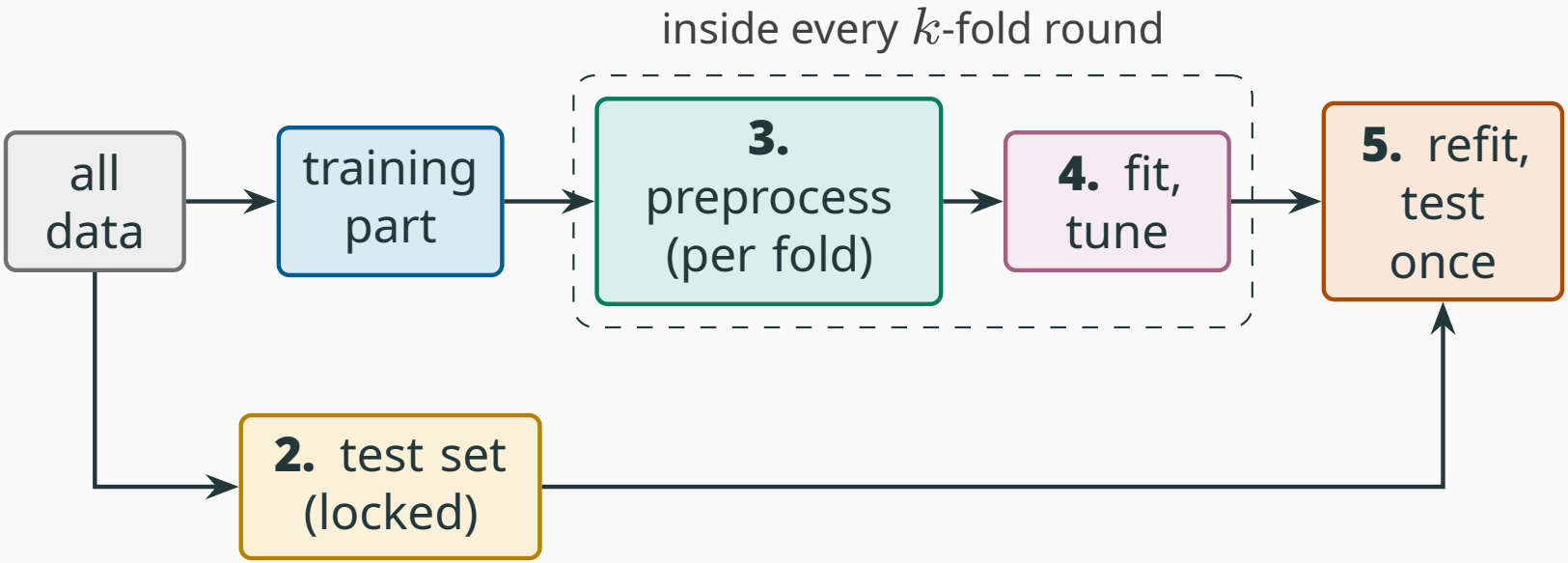
- Squared error is dominated by the **largest values**: one district with 1243 people per house outweighs thousands of ordinary ones
- A logarithm turns *ratios* into *differences* and pulls the tail in ($\log(1+x)$ keeps 0 at 0)
- Housing, least squares, test MSE: **0.497** raw, **0.421** with the log (R^2 0.623 to 0.681)
- Spam, logistic regression, 5-fold MCC: **0.831** raw, **0.877** with $\log(1+x)$

Preprocessing: Missing Values, Categories

Problem	What to do	Learned from
Missing number	impute the **median** (+ a 0/1 "was missing" column if absence means something)	training rows
Category	**one-hot**: a 0/1 column per level; an unseen level gives zeros	training levels
Many levels	group the rare ones; target encoding needs nested folds	training folds

- **Standard steps of the preprocessing toolset**, like scaling: `SimpleImputer`, `OneHotEncoder`, `ColumnTransformer` (scikit-learn); `fill_null`, `to_dummies` (polars)
- Dropping incomplete rows changes the **population** you evaluate on: say so, or impute
- The median of **all** rows is the same leak as scaling with all rows

The Pipeline, End to End



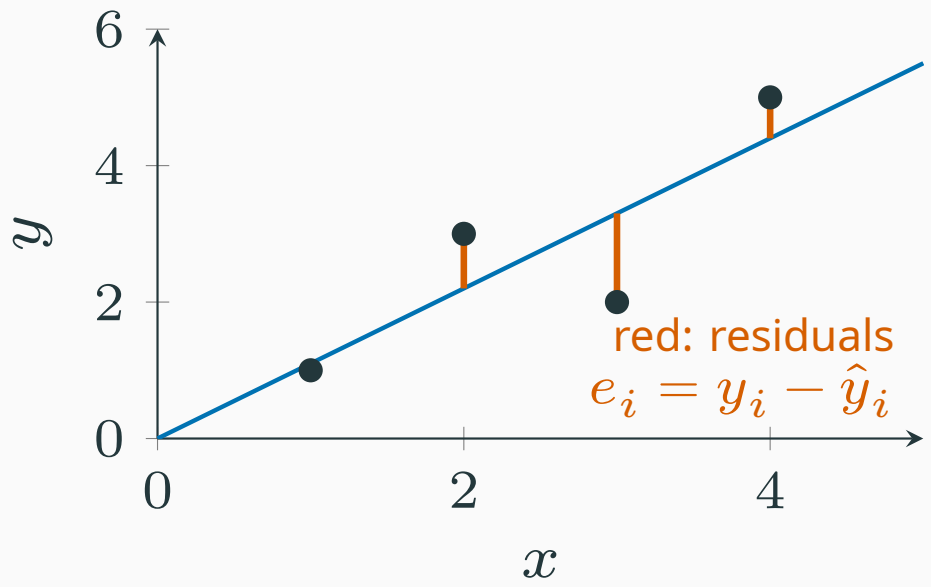
1. **Data visualization:** decides the transformations and the metric
2. **Evaluation methodology:** split once, keep the test set **locked**, choose folds and metric
3. **Preprocessing:** fitted on the training folds only (scale, log, impute, encode)
4. **Model:** fit and tune on validation folds
5. **Report:** refit, test **once**, with baseline and spread

DEMO R1:

`01_linear_regression.ipynb`

Linear Regression

The Model: a Hyperplane

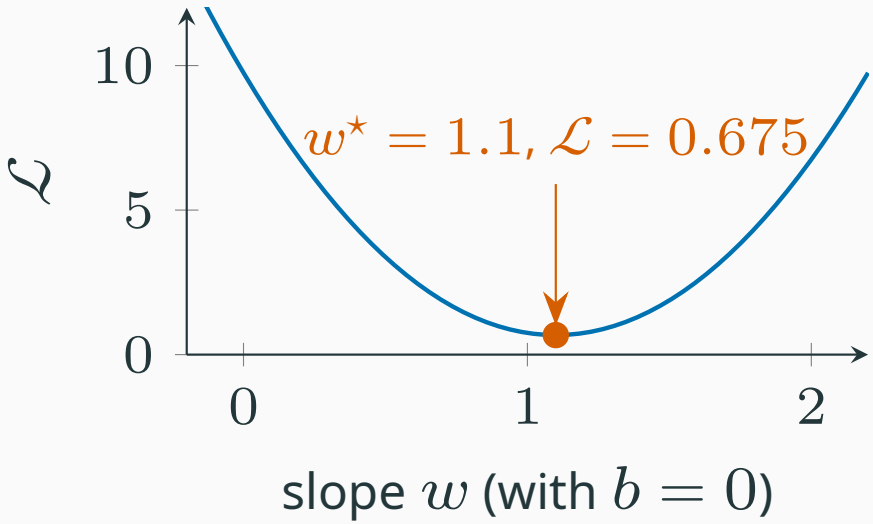


$$\hat{y} = w^\top x + b = w_1 x_1 + \cdots + w_d x_d + b$$

- d weights and one bias: $d + 1$ numbers **are** the model
- Each weight: the change of $\hat{y}$ per unit of feature j, the others fixed
- Housing: $d = 8$; with standardized inputs the weights are comparable

The Loss: Mean Squared Error

$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^{N} \left(w^\top x_i + b - y_i \right)^2$$



- A **quadratic** in (w, b): a convex bowl with one minimum (Class 02)
- Squaring: errors cannot cancel, large errors cost more
- Same four points as the previous slide

Worked Example: Least Squares by Hand

Points $(1,1), (2,3), (3,2), (4,5)$, so $\bar{x} = 2.5$ and $\bar{y} = 2.75$

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{5.5}{5} = 1.1, \qquad b = \bar{y} - w\bar{x} = 0$$

x	y	$\hat{y}$	e
1	1	1.1	-0.1
2	3	2.2	0.8
3	2	3.3	-1.3
4	5	4.4	0.6

$$\text{MSE} = \frac{0.01 + 0.64 + 1.69 + 0.36}{4} = 0.675$$

The Gradient and the Normal Equation

$$\nabla_w \mathcal{L} = \frac{2}{N} X^\top (Xw + b - y) \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{2}{N} \sum_i (\hat{y}_i - y_i)$$

- Gradient = features $\times$ residuals: each weight moves to explain the residual it can see
- Set it to zero. With $A = [X \ \mathbf{1}]$ and $\theta = (w, b)$:

$$A^\top A\, \theta = A^\top y \quad \Longrightarrow \quad \theta^\star = (A^\top A)^{-1} A^\top y$$

- The **normal equations**: a linear system. Solve it (`np.linalg.solve`), never invert
- Exists whenever $A^\top A$ is invertible: no perfectly collinear features, and $N \geq d + 1$
- In the lab: **you never type this gradient**; `jax.grad` of the loss gives it

Worked Example: Gradient Descent

Same four points, start $w = b = 0$, $\eta = 0.05$

step	w	b	$\mathcal{L}$	$\partial_w \mathcal{L}$	$\partial_b \mathcal{L}$
0	0.000	0.000	9.750	-16.500	-5.500
1	0.825	0.275	0.940	-2.750	-0.825
2	0.963	0.316	0.699	-0.481	-0.055
3	0.987	0.319	0.692	-0.107	0.071

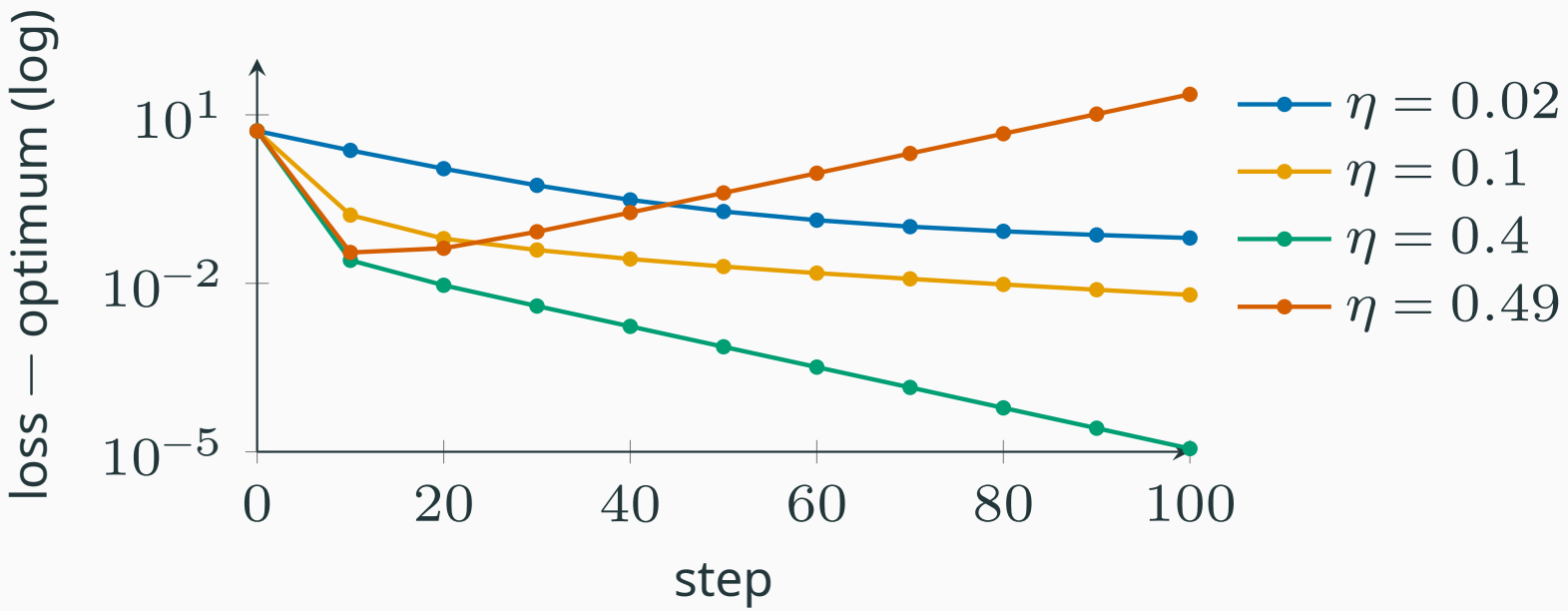
- Step 0:
  $\partial_w \mathcal{L} = \frac{2}{4} \sum_i (0 - y_i) x_i = -0.5 \cdot 33 = -16.5$, so
  $w_1 = 0 - 0.05 \cdot (-16.5) = 0.825$
- Heading for $(w, b) = (1.1, 0)$ and $\mathcal{L} = 0.675$: the answer of the closed form
- Linear regression is the one model where we can **check** the iterative answer against the exact one

Closed Form or Gradient Descent?

	Normal equation	Gradient descent
Cost	$O(Nd^2 + d^3)$ once	$O(Nd)$ per step, many steps
Needs	$A^\top A$ invertible	any differentiable loss
Tuning	none	learning rate, steps
Large d	hopeless (10^6 features)	works (mini-batches, Class 02)
Other losses (Huber, L1, ...)	no closed form	change one line

- For 8 housing features the closed form is instant; **all other models today have no closed form** (or only a variant)
- Both give the same weights: difference $6 \cdot 10^{-15}$ (normal equation vs scikit-learn), $2 \cdot 10^{-6}$ (300 GD steps)

The Learning Rate Comes from the Curvature



- The Hessian of the MSE is constant: $H = \frac{2}{N} A^\top A$. Housing: eigenvalues **0.102** to **4.166**, condition number **41**
- Stable only for $\eta < 2/\lambda_{\max} = 0.48$: $\eta = 0.49$ diverges, $\eta = 0.02$ crawls along the flat direction (Class 02)

Linear Regression in JAX

```python
import jax, jax.numpy as jnp

def predict(params, X):
    return X @ params["w"] + params["b"]

def mse_loss(params, X, y):
    return jnp.mean((predict(params, X) - y) ** 2)

params = {"w": jnp.zeros(8), "b": jnp.zeros(())}
grad = jax.jit(jax.grad(mse_loss))       # derived, not typed
for _ in range(300):
    g = grad(params, X_train, y_train)
    params = jax.tree.map(lambda p, gp: p - 0.4 * gp, params, g)
```

- Parameters are a **pytree** (a dictionary): `jax.grad` returns the gradient with the same structure
- Change the loss (add a penalty, use Huber) and **nothing else** changes: this is the lab's method

Why Squared Error? The Probabilistic View

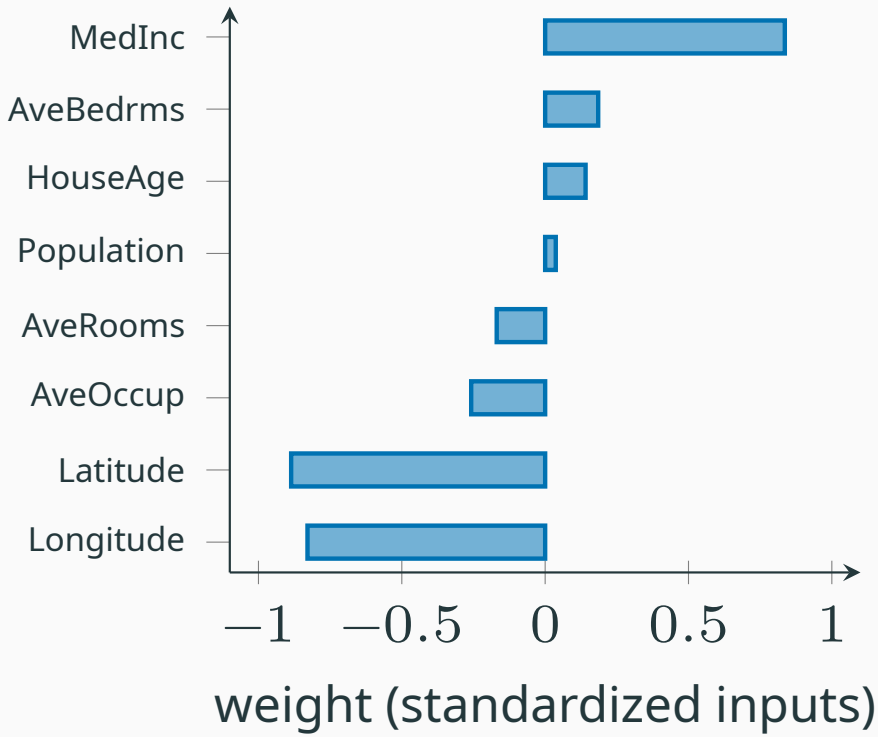
$$y = w^\top x + b + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\Longrightarrow \quad p(y \mid x) = \mathcal{N}\left(w^\top x + b,\ \sigma^2\right)$$

$$-\log \prod_i p(y_i \mid x_i) = \frac{1}{2\sigma^2} \sum_i (y_i - \hat{y}_i)^2 + N \log \sigma + \text{const}$$

- Minimizing the MSE is **maximum likelihood** under Gaussian noise (Class 01: the loss is a code length)
- Laplace noise gives the absolute error (MAE, robust to outliers); Gaussian noise punishes outliers heavily
- The same idea gives **cross-entropy** for classes (a few slides ahead)

Result: House Prices



Test set (4128 districts)

	RMSE	R^2
mean baseline	1.148	0.000
linear	**0.649**	**0.681**

RMSE 0.649 = **64 900 USD** typical error

- Income raises the price; latitude and longitude (the coast) dominate the rest
- About 32% of the variance is unexplained: what a hyperplane cannot see (and the cap at 5.0)

Live Demo: Linear Regression Pipeline

DEMO R2:

`01_linear_regression.ipynb`

Regression Metrics

How Wrong Are the Predictions?

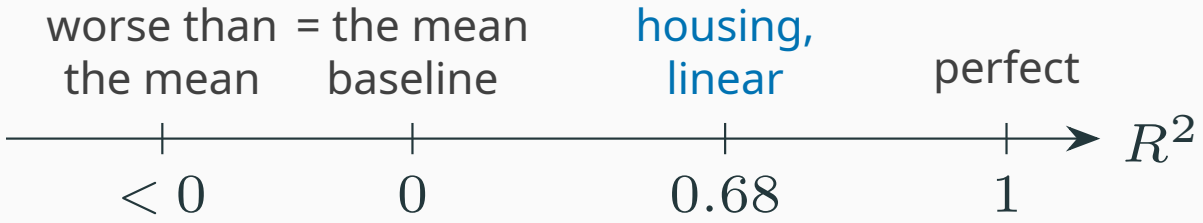
$$e_i = y_i - \hat{y}_i \qquad \text{(the residual)}$$

$$\text{MSE} = \frac{1}{n}\sum_i e_i^2 \qquad \text{RMSE} = \sqrt{\text{MSE}} \qquad \text{MAE} = \frac{1}{n}\sum_i |e_i|$$

- **MSE:** the loss we optimize; squared unit, punishes large errors
- **RMSE:** back in the unit of y, dominated by the large errors
- **MAE:** every error counts in proportion; robust to outliers
- Always **RMSE** $\geq$ **MAE**: the gap measures how uneven the errors are
- Lower is better; none is meaningful without a **baseline**

R^2: Compared with Predicting the Mean

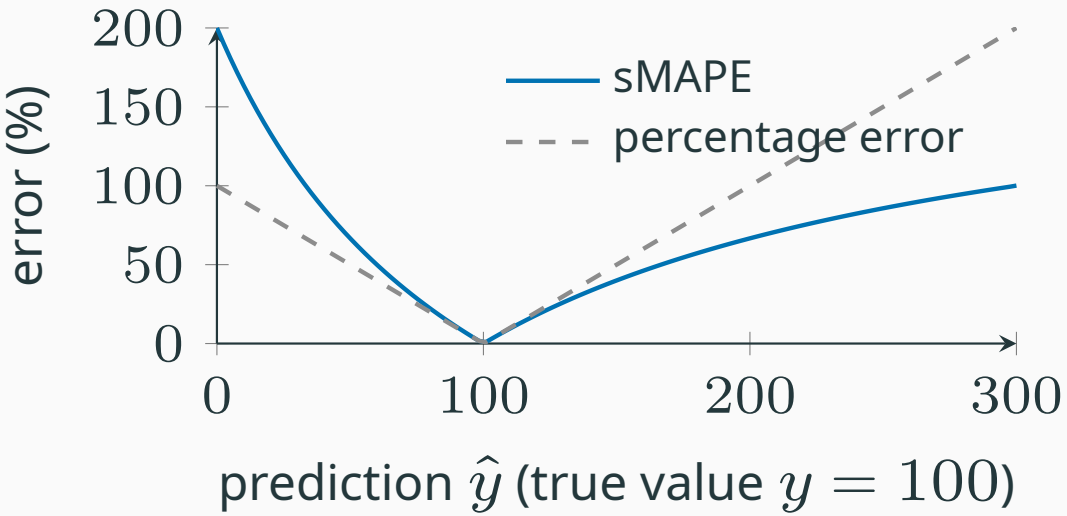
$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\text{MSE of the model}}{\text{MSE of the mean baseline}}$$



- **Fraction of the variance explained** by the model; 0 is the mean baseline, 1 is perfect
- It can be **negative**: a constant prediction of 10 for values near 3 gives $R^2 = -98$
- Scale-free, but never enough alone: it says nothing about the size of the errors in USD

sMAPE: a Symmetric Relative Error

$$\text{sMAPE} = \frac{100}{n} \sum_i \frac{2\,|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|} \quad [\%]$$



- True value 100: prediction 50 gives **66.7%**, prediction 150 gives **40.0%** (same error, plain percentage error 50% for both)
- So it punishes **under-prediction** more; unstable near 0: use it for prices

Worked Example: the Four Metrics

Residuals of the least-squares line:
$e = (-0.1,\ 0.8,\ -1.3,\ 0.6),\ y = (1, 3, 2, 5),\ \bar{y} = 2.75$

Metric	Computation	Value
MSE	$(0.01 + 0.64 + 1.69 + 0.36)/4$	0.675
RMSE	$\sqrt{0.675}$	0.822
MAE	$(0.1 + 0.8 + 1.3 + 0.6)/4$	0.700
R^2	$1 - 2.7/8.75$	0.691
sMAPE	mean of $\frac{2\lvert e_i\rvert}{y_i + \hat{y}_i}$: $9.5\%,\ 30.8\%,\ 49.1\%,\ 12.8\%$	25.5%

- $\sum (y_i - \bar{y})^2 = 3.0625 + 0.0625 + 0.5625 + 5.0625 = 8.75$
- Four numbers, four questions: how big (RMSE), how typical (MAE), how much explained (R^2), how relative (sMAPE)

Metrics Disagree: Outliers

Predictions	MSE	RMSE	MAE	sMAPE	R^2
errors ± 0.1	0.010	0.100	0.100	3.5%	0.98
same, **one** error of $+3$	1.93	1.39	0.70	16.5%	-2.86

- True values $2, 2.5, 3, 3.5, 4$; a single prediction 3 units off
- **MSE** $\times$ **193**, MAE $\times$ 7: squaring lets one point dominate
- Choose by the **cost of a mistake**: quadratic cost (safety margins) $\Rightarrow$ MSE/RMSE; linear cost, dirty data $\Rightarrow$ MAE
- Report at least one absolute and one relative measure, plus the baseline

Baselines: Mean, Median, Mode

Constant answer	Value	MSE	MAE	sMAPE	R^2
mean	2.069	**1.319**	0.904	44.6%	0.000
median	1.797	1.392	**0.875**	**43.4%**	-0.056
mode	5.000	9.915	2.932	89.8%	-6.520
linear regression		*0.421*	*0.472*	*25.6%*	*0.681*

- The **mean** minimizes the squared error, the **median** the absolute error: each wins *its own* metric, so the baseline must be chosen with the metric
- The **mode** is the most frequent value: here the cap at 5.0, a terrible constant for a continuous target
- A model is only worth reporting if it beats the best baseline **for the metric you report**

Result: House Prices

Model	MSE	RMSE	MAE	sMAPE	R^2
mean baseline	1.319	1.148	0.904	44.6%	0.000
linear regression	**0.421**	**0.649**	**0.472**	**25.6%**	**0.681**

- Units: 100 000 USD. The typical error falls from **114 800** USD to **64 900** USD
- RMSE (0.649) above MAE (0.472): a few districts are badly wrong (the capped ones, the very expensive)
- sMAPE 25.6%: a quarter of the price, on average. Useful, not precise: the next slides try to do better

DEMO R3:

`01_linear_regression.ipynb`

Polynomial Feature Expansion

Still Linear, in the Parameters

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3 = w_0 + w^\top \phi(x)$$

$$\phi(x) = (x,\ x^2,\ x^3)$$

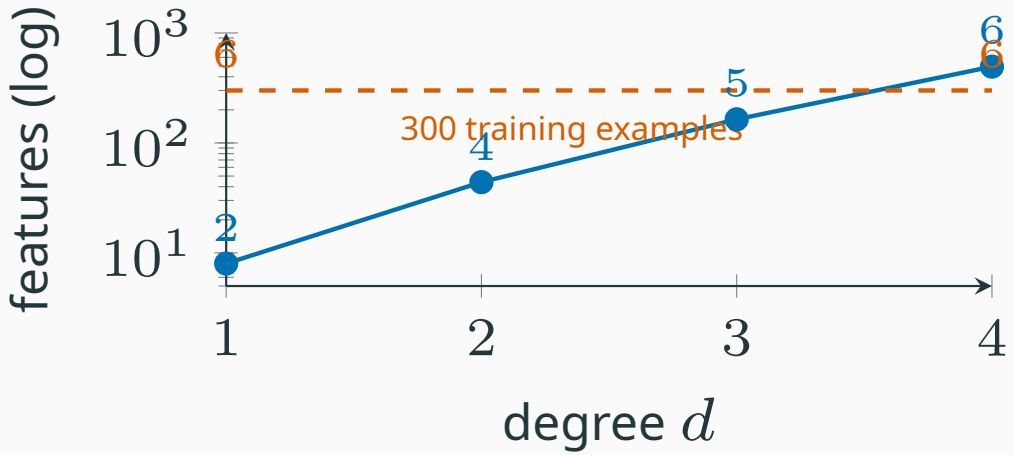
- The model is a **curve** in x but a **hyperplane** in the new features $\phi(x)$
- The loss is still a quadratic in w: the same normal equation, the same `jax.grad`, the same metrics
- With several inputs: **all monomials** up to degree d, e.g. degree 2 for (x_1, x_2):

$$\phi(x_1, x_2) = (x_1,\ x_2,\ x_1^2,\ x_1 x_2,\ x_2^2)$$

- The products $x_1 x_2$ are **interactions**: the effect of one feature depends on another

The Number of Features Explodes

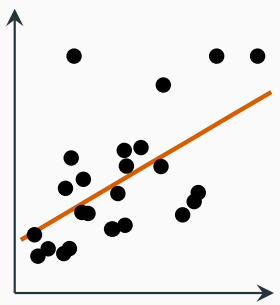
$$\#\text{features} = \binom{n+d}{d} - 1 \qquad (n \text{ inputs, degree } d)$$



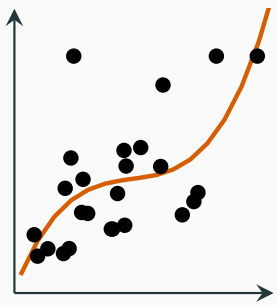
- The 8 housing features: 8, **44**, **164**, **494** columns (degrees 1 to 4)
- Beyond N columns the model **interpolates** the training set
- **Standardize after expanding** (x^4 has a very different scale)

Curves of Growing Flexibility

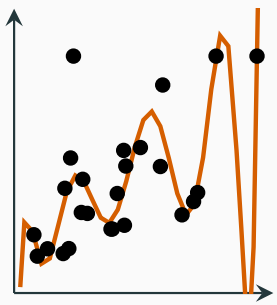
degree 1: underfits



degree 3: smooth



degree 9: memorizes



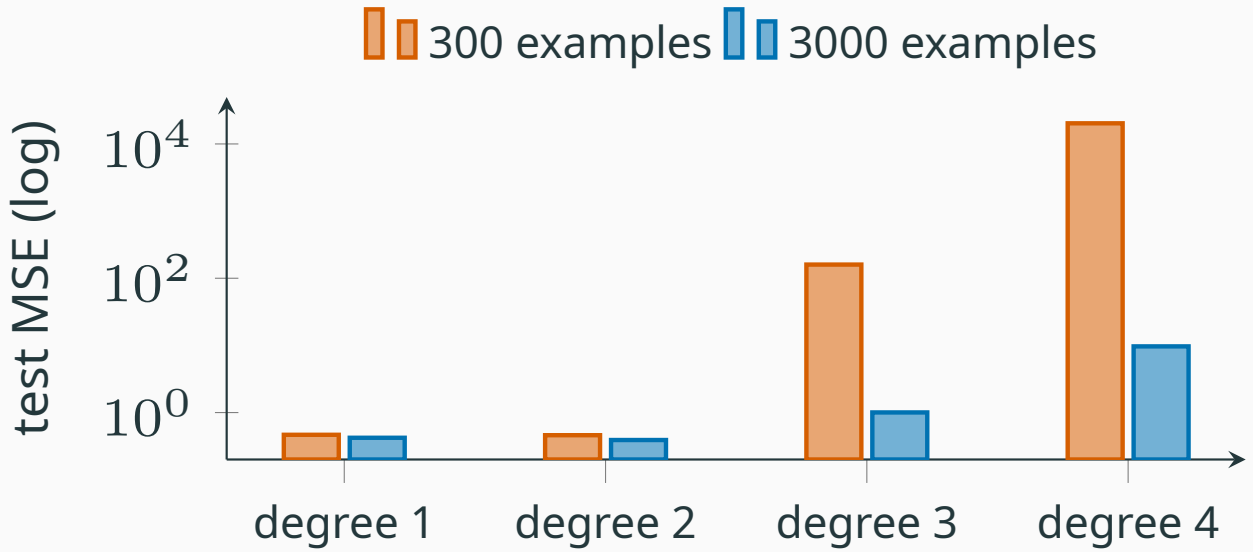
- 25 districts: value against (standardized) median income, fitted with polynomials of degree 1, 3 and 9
- Degree 1 misses the bend, degree 9 chases the noise and swings wildly at the edges

Training Error Versus Test Error

degree	features	train MSE	test MSE
1	8	0.478	**0.467**
2	44	0.310	**0.460**
3	164	0.135	159.8
4	494	0.0003	20 289

- 300 training examples of the 8 housing features; the **training** error falls at every degree
- At degree 4 there are more features than examples: the fit is perfect on the training set, useless on the test set
- Degree 3 is *worse* than degree 2 by a factor of **350**: outlying districts, cubed, dominate
- The **test** (or validation) error picks the degree, never the training error

More Data Tames a Flexible Model



- Ten times more data: degree 3 goes from **160** to **1.0**, degree 4 from **20 000** to **9.7**
- Flexibility only pays when the data can *support* it: at 3000 examples degree 2 (**0.390**) finally beats the line (**0.421**)
- The other cure keeps the data and **constrains the model**: regularization

Live Demo: Polynomial Features

DEMO R4:

`01_linear_regression.ipynb`

Regularization

Penalize Large Weights

$$\min_{w,\,b} \underbrace{\frac{1}{N}\sum_i (\hat{y}_i - y_i)^2}_{\text{fit the data}} + \lambda \underbrace{\Omega(w)}_{\text{keep the weights small}}$$

	$\Omega(w)$	Name	Effect
L_2	$\|w\|_2^2 = \sum_j w_j^2$	**ridge** (L_2 regularization, weight decay)	shrinks all weights smoothly
L_1	$\|w\|_1 = \sum_j \|w_j\|$	**lasso** (L_1 regularization)	shrinks and sets many to **exactly 0**
both	$\rho\|w\|_1 + \frac{1-\rho}{2}\|w\|_2^2$	**elastic net**	sparse and stable

- $\lambda \geq 0$ is a **hyper-parameter**: 0 is least squares, ∞ predicts the mean
- The bias b is **not** penalized; **standardize first**, the

Ridge (L_2) Regression

$$\mathcal{L}_{ridge} = \mathcal{L}_{L_2} = \frac{1}{N}\|y - Xw - b\|^2 + \lambda\|w\|_2^2$$

$$w^\star = (X^\top X + \lambda N I)^{-1} X^\top y \qquad \text{(centred data)}$$

- Adding $\lambda N I$ to $X^\top X$ makes it **invertible and well conditioned**: highly correlated polynomial features no longer blow up
- Gradient descent: the penalty adds $2\lambda w$ to the gradient, a "weight decay" towards 0
- One feature, centred: $w_{ridge} = \dfrac{w_{ols}}{1 + \lambda N / \sum x_i^2}$, a **shrinkage factor** below 1
- $\lambda = 0$: least squares; moderate λ: smooth and stable; $\lambda \to \infty$: $w \to 0$, the mean (more bias, less variance)

Lasso (L_1) and Elastic Net

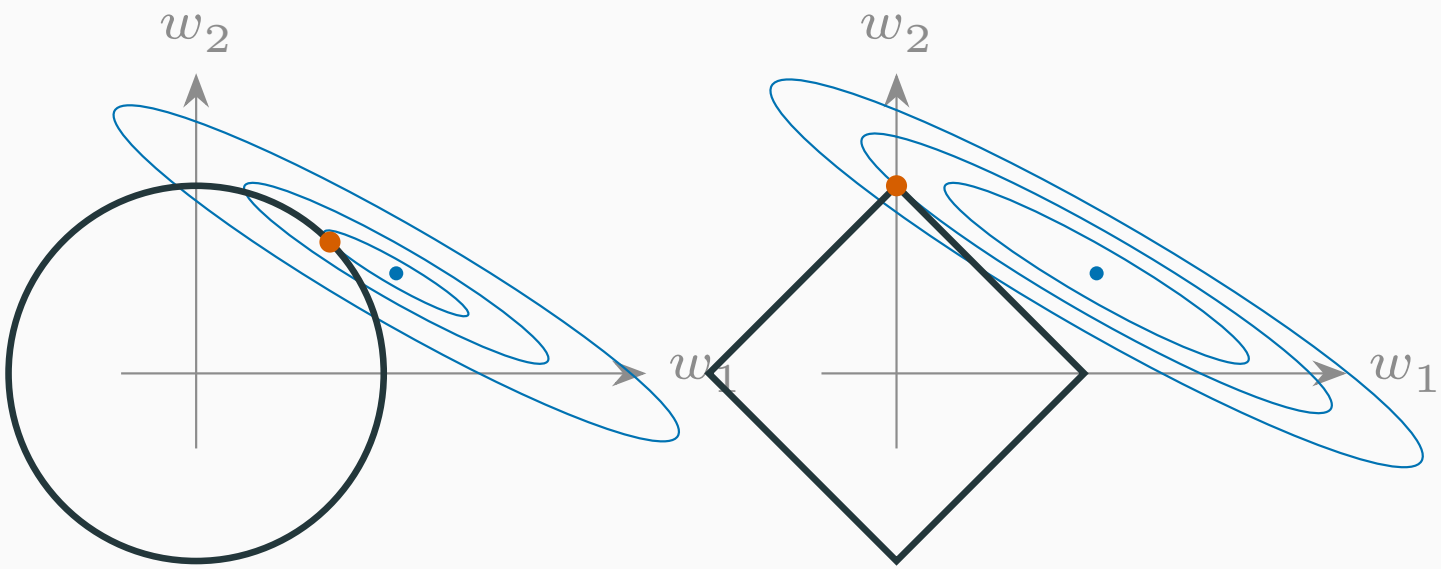
$$\mathcal{L}_{lasso} = \mathcal{L}_{L_1} = \frac{1}{N}\|y - Xw - b\|^2 + \lambda\|w\|_1$$

- No closed form ($|w|$ has a kink at 0), but still **convex**: coordinate descent (scikit-learn) or a **proximal** step, descend on the MSE, then **soft-threshold**

$$\text{soft}(w, t) = \text{sign}(w)\ \max(|w| - t,\ 0)$$

- A weight pulled less by the data than by the penalty **stays at 0**: **feature selection**
- `jax.grad` of $|w|$ is a *subgradient*: weights hover near 0, never exactly; the proximal step gives exact zeros (lab E3)
- **Elastic net** keeps correlated features together; lasso picks one

The Geometry: Why L_1 Gives Zeros

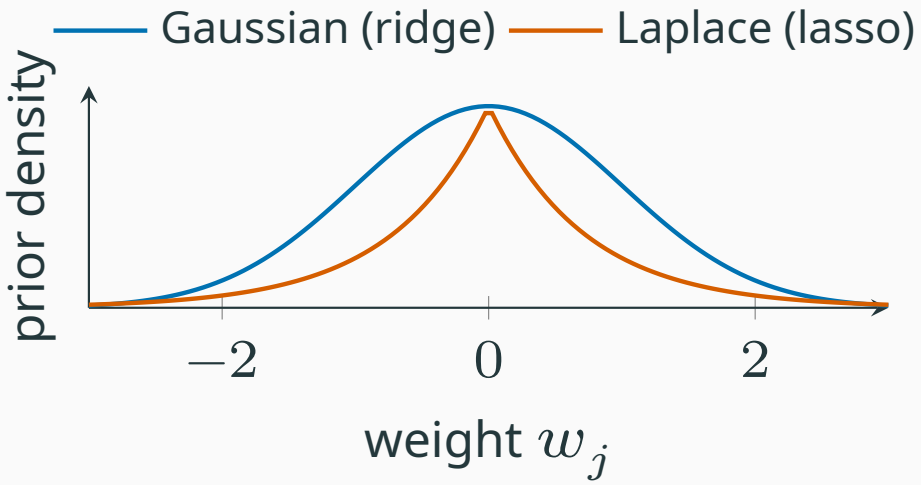


ridge (L_2): contact off the axes (both weights non-zero)

lasso (L_1): contact at a corner ($w_1 = 0$ exactly)

- Blue: contours of the squared error (centre = least squares). Black: the region $\|w\|_p \leq t$; the solution is where they touch
- The L_1 region has **corners on the axes**: contours tend to meet it there

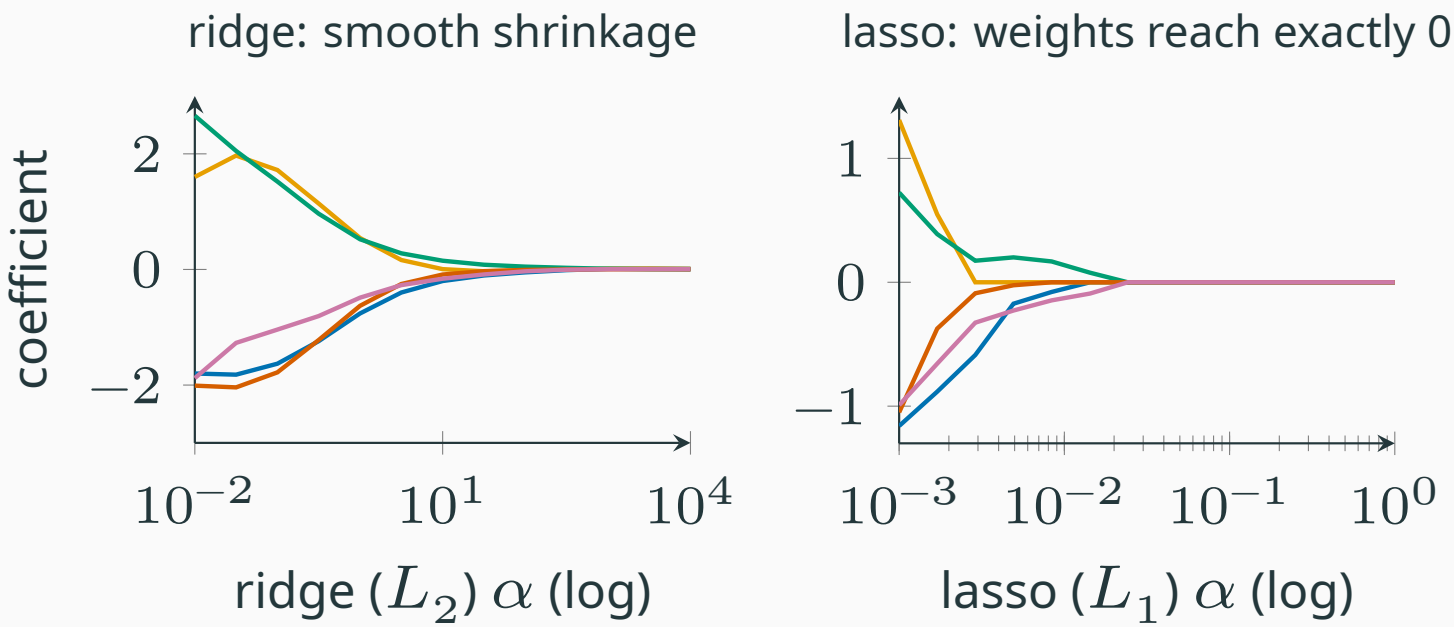
The Prior View: MAP



$$\hat{w}_{MAP} = \arg\min_{w} \underbrace{-\log p(y \mid X, w)}_{\text{data: MSE}} \underbrace{- \log p(w)}_{\text{prior: penalty}}$$

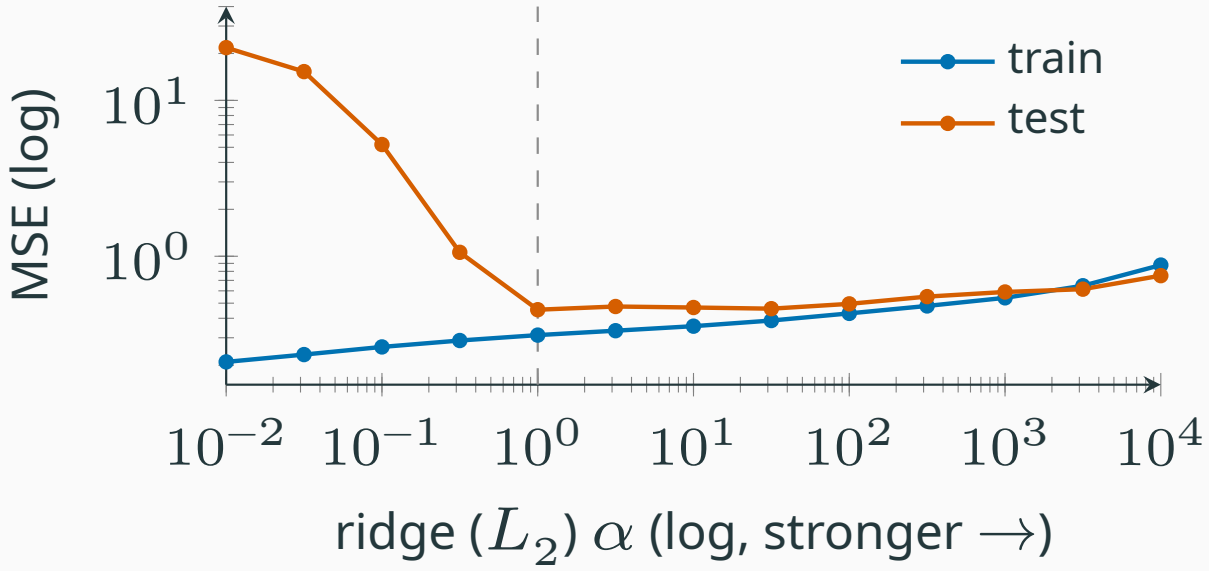
- Gaussian prior $\Rightarrow L_2$; Laplace prior $\Rightarrow L_1$ (a peak at 0: most weights are 0)
- Class 01 (MDL): the penalty is the **code length of the model**, the MSE that of the errors
- λN = noise variance / prior variance

Regularization Paths



- 300 examples, 164 degree-3 features, five coefficients; regularization is **stronger to the right** in both plots
- Ridge: every weight shrinks smoothly. Lasso: weights become exactly 0 one after the other; non-zero weights: $\alpha = 1$: 0 of 164, 0.07: 7, 0.014: 20, 0.001: 44

Choosing λ



- A U-shaped **validation** curve: too small α overfits (test 21.8), too large underfits (test 0.75); training error rises monotonically
- Choose λ on a **validation set or by cross-validation** (`RidgeCV`, `LassoCV`); the test set is touched once
- Search on a **logarithmic grid** ($10^{-4}, \dots, 10^{4}$); refine around the best value

Conventions: λ and scikit-learn

Model	scikit-learn minimizes	Our λ (MSE = a mean)
Ridge	$\|y - Xw\|^2 + \alpha\|w\|^2$	$\alpha = \lambda N$
Lasso	$\frac{1}{2N}\|y - Xw\|^2 + \alpha\|w\|_1$	$\alpha = \lambda/2$
Logistic	$C\sum_i \ell_i + \frac{1}{2}\|w\|^2$	$C = 1/(\lambda N)$ (loss $+ \frac{\lambda}{2}\|w\|^2$)

- The same idea, three scalings: a mismatch is the usual reason two "identical" models disagree
- Lab: after mapping λ, our ridge equals `Ridge` to $3 \cdot 10^{-16}$ and our logistic equals `LogisticRegression` to $2 \cdot 10^{-6}$
- Lasso weights differ from scikit-learn's by up to 0.15–0.20 while the **objective differs by** 10^{-5}: nearly collinear features have many equally good solutions

Result: Taming a Degree-3 Model

Model (164 features, 300 examples)	Test MSE	Non-zero weights
plain least squares	159.8	164
ridge ($\alpha = 1$, best on test)	**0.454**	164
ridge, α by 5-fold CV (31.6)	0.461	164
lasso ($\alpha = 0.0203$, best on test)	0.494	**15**
lasso, α by 5-fold CV (0.01)	0.547	19
elastic net ($\rho = 0.5$)	0.535	39
(for reference) the plain line, 8 features	*0.467*	*8*

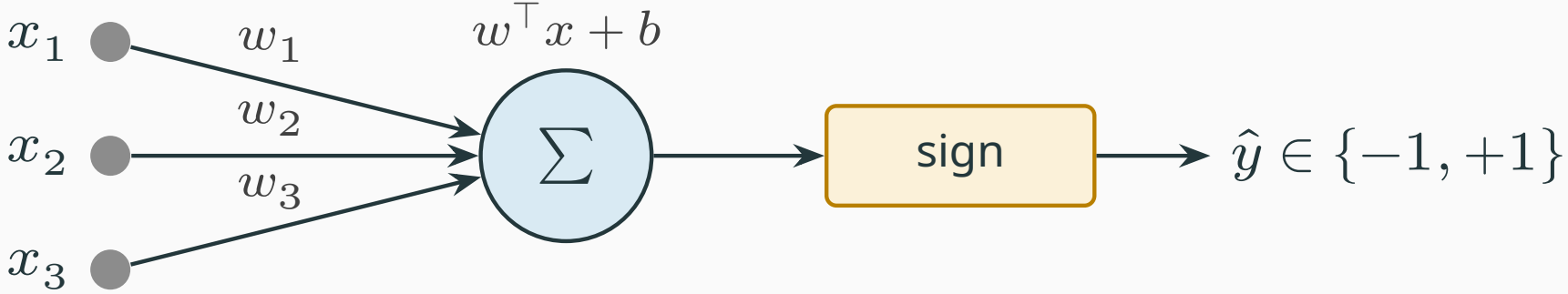
- Regularization turns a disastrous model into a good one; the lasso does it with **15** of 164 features
- Choosing α on the test set (the "best" rows) is optimistic: the CV rows are the honest numbers

DEMO R5:

`01_linear_regression.ipynb`

The Perceptron

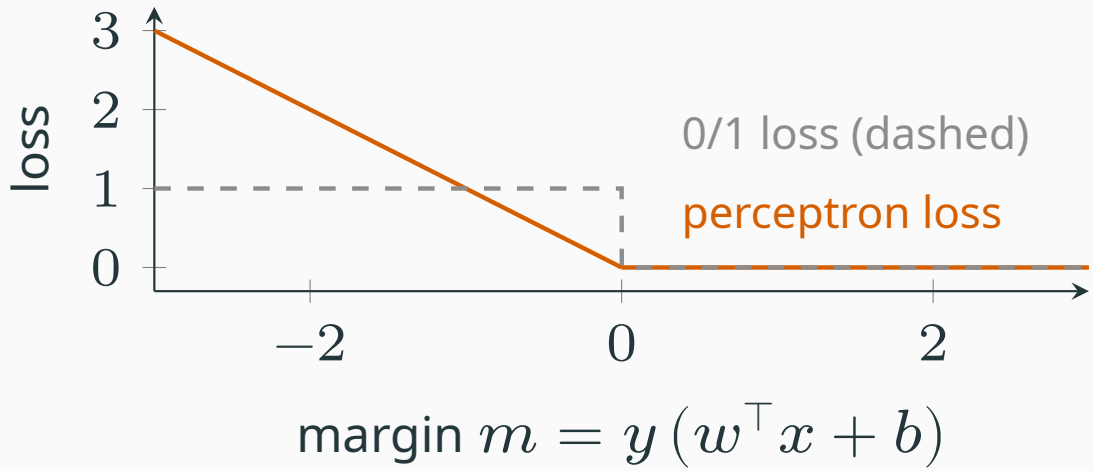
A Neuron That Draws a Line



$$\hat{y} = \text{sign}(w^\top x + b) \qquad y \in \{-1, +1\}$$

- The first learning machine (Rosenblatt, 1958): a weighted vote, then a hard decision
- The **decision boundary** $w^\top x + b = 0$ is a hyperplane; w is its normal, pointing to the $+1$ side
- The **margin** $m = y\,(w^\top x + b)$ is positive when the point is correctly classified
- It is the building block of neural networks (Class 05)

The Perceptron Loss and Its Update



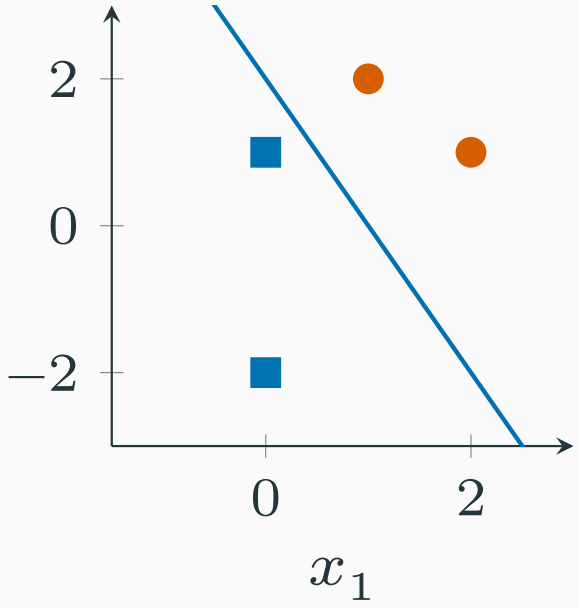
$$\ell = \max(0,\, -m), \qquad \nabla_w \ell = \begin{cases} -y\,x & m \le 0 \text{ (a mistake)} \\ 0 & m > 0 \end{cases}$$

- A gradient step of length 1: on a mistake, $w \leftarrow w + y\,x$ and $b \leftarrow b + y$; otherwise nothing
- **The perceptron rule is a subgradient step** on this loss: in the lab, `jax.grad` of one example *is* the update

Worked Example: the Perceptron by Hand

Points $(2,1)^+$, $(1,2)^+$, $(0,-2)^-$, $(0,1)^-$; $w = (0,0)$, $b = 0$

epoch	point	y	score	new w	new b
1	$(2,1)$	$+$	0	$(2,1)$	1
1	$(0,1)$	$-$	2	$(2,0)$	0
2	$(0,-2)$	$-$	0	$(2,2)$	-1
2	$(0,1)$	$-$	1	$(2,1)$	-2
3	all four			no change	



- Only the mistakes are listed (a **score** of 0 counts as a mistake); the other visits change nothing
- Final boundary $2x_1 + x_2 - 2 = 0$: all four points are correct, so the algorithm stops

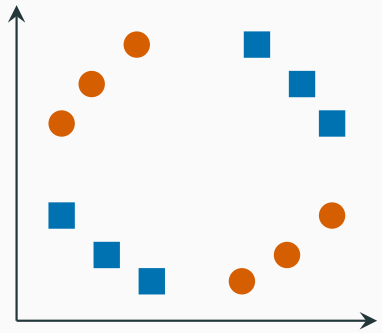
Convergence, and When It Fails

Data	Mistakes per epoch	Outcome
separable blobs	2, 0, 0, 0, …	**converges**
overlapping blobs	20, 14, 14, 14, 11, 14, 10, 14, …	never stops
XOR, raw features	98, 100, 98, 92, 96, …	accuracy 0.42
XOR, degree-2 features	16, 2, 0, 0, …	**converges**

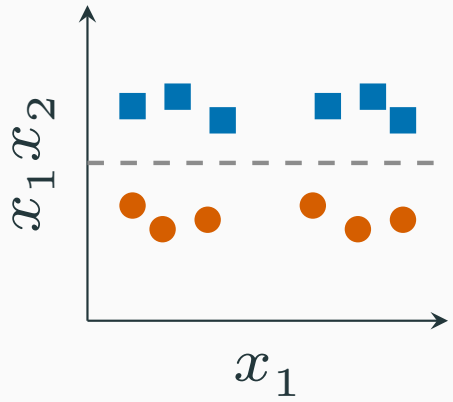
- **Novikoff (1962):** on linearly separable data with margin γ and $\|x\| \le R$, at most $(R/\gamma)^2$ updates, whatever the order
- On overlapping data it cycles: the final weights are arbitrary, and there is no "best" line in sight
- The exact behaviour is shared with scikit-learn: our `jax.grad` perceptron and `Perceptron(penalty=None, eta0=1, shuffle=False)` give **identical** weights (difference **0.0**)

XOR: Beyond a Straight Line

raw (x_1, x_2): no line works



with $x_1 x_2$: a threshold works



- XOR: class 1 when the coordinates have **different signs** (Minsky and Papert, 1969)
- Raw features: chance level (accuracy **0.42**); adding $x_1 x_2$ makes it **separable**
- Same for logistic regression: accuracy 0.655 on raw XOR, **1.0** with degree-2 features (lab)
- Learning the features instead is the hidden layer (Class 05)

The Perceptron on Spam

- One pass over the 3680 training e-mails (log frequencies, standardized), a step of 1:
  - **394 mistakes** in the epoch (10.7%)
  - test accuracy **0.912**
- Fast, online, no probabilities, no learning rate to tune
- Sensitive to the **order** of the examples and to feature scale (a large feature dominates the update)
- No notion of *how confident* a prediction is: a point just inside the boundary looks the same as one far away
- Its smoother cousin, next: **logistic regression**, which turns the same score $w^\top x + b$ into a probability

DEMO P1:

`02_perceptron.ipynb`

Generalized Linear Models

Generalized Linear Models: Three Parts

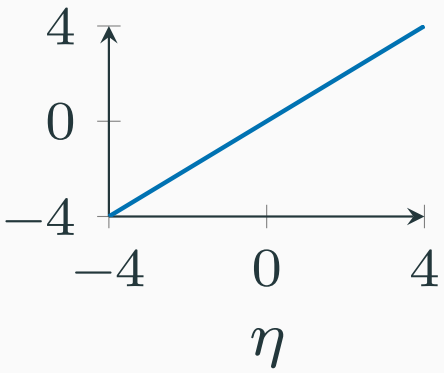
$$g\big(\mathbb{E}[y \mid x]\big) = w^\top x + b$$

Part	Role	Linear regression
linear predictor	$\eta = w^\top x + b$: what we learn	the same
link g	connects the mean of y to η	identity
distribution of y	how y varies around its mean (exponential family)	Gaussian

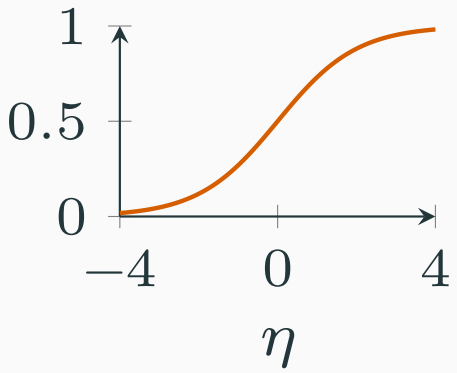
- We already know one member: linear regression is a Gaussian y with the identity link
- A **GLM** keeps the linear predictor and changes the other two parts to fit the *type* of y: a probability, a count, a class
- The loss is always the **negative log-likelihood** of the chosen distribution (Class 04 derives it)

Why a Link? The Mean Must Stay in Its Range

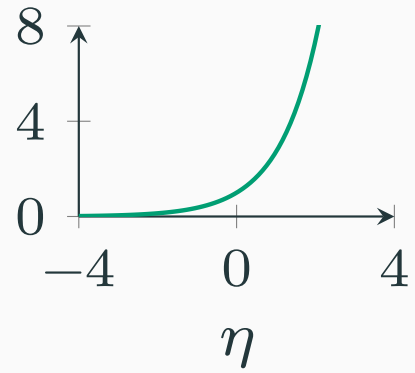
identity: $\mu = \eta$



logit link: $\mu = \sigma(\eta)$



log link: $\mu = e^{\eta}$



Model	$y \mid x$	Link g	Mean μ lives in
Linear regression	Gaussian	identity	$\mathbb{R}$
Logistic regression	Bernoulli	logit	$(0, 1)$
Poisson regression	Poisson	log	$(0, \infty)$

- The score η is any real number; the **inverse link** maps it to a valid mean (a probability, a rate)

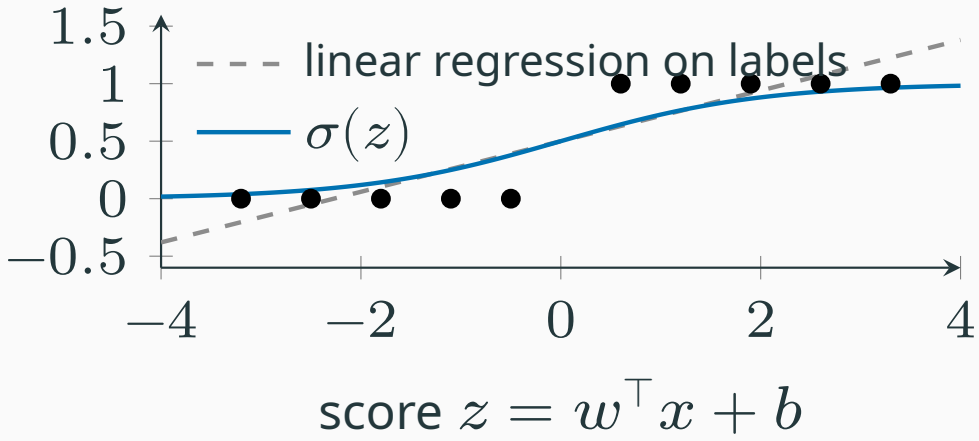
The Canonical Link: One Gradient for the Family

$$\mathcal{L} = -\tfrac{1}{N} \sum_i \log p(y_i \mid x_i) \quad \Rightarrow \quad \nabla_w \mathcal{L} = \tfrac{1}{N} X^\top (\hat{\mu} - y)$$

- With the **canonical link** (identity, logit, log) the gradient is always *features* $\times$ *residual*: the same JAX code serves the whole family
- Linear regression: $\hat{\mu} = \hat{y}$, residual $\hat{y} - y$ (the gradient seen before); convex in w for every canonical-link GLM
- scikit-learn: `PoissonRegressor`, `GammaRegressor`, `TweedieRegressor` (and the two we know)
- The **perceptron** has no probability model: it is *not* a GLM

Logistic Regression

From a Score to a Probability



$$p(y = 1 \mid x) = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

- A line fitted to 0/1 labels leaves $[0, 1]$: not a probability (schematic data)
- The GLM for a yes/no y: **Bernoulli** distribution with the **logit link**
- The sigmoid maps any score into $(0, 1)$; the boundary $p = 0.5$ is still $w^\top x + b = 0$; the **threshold** is a choice

Odds and Log-Odds

p	odds $\frac{p}{1-p}$	log-odds z
0.01	0.010	-4.60
0.10	0.111	-2.20
0.50	1	0
0.90	9	2.20
0.99	99	4.60

$$\log \frac{p}{1-p} = w^\top x + b$$

- The model is **linear in the log-odds**: each weight adds evidence for or against spam
- e^{w_j} is the **odds ratio**: the odds get multiplied by e^{w_j} when feature j grows by one standard deviation
- Spam: `char_freq_!` $+0.74$ ($\times 2.09$). Ham: `word_freq_hp` -1.37 ($\times 0.25$)

Cross-Entropy: Maximum Likelihood for Labels

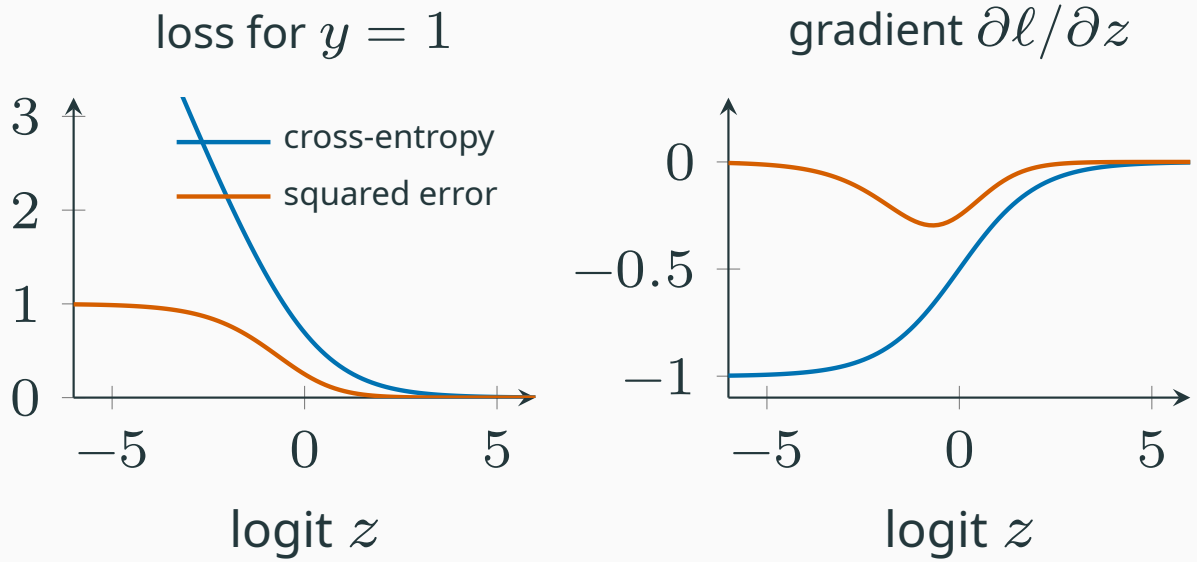
Each label is a Bernoulli variable with $P(y=1)=p_i$; the likelihood of the data and its negative log:

$$-\log \prod_i p_i^{y_i}(1-p_i)^{1-y_i} = \sum_i \ell_i, \qquad \ell_i = -y_i \log p_i - (1-y_i)$$

$$\ell_i = \log\left(1+e^{z_i}\right) - y_i z_i \qquad (z_i = w^\top x_i + b)$$

- The **binary cross-entropy**: the code length of the labels (Class 01)
- With the logit z and `jnp.logaddexp(0, z)` it never overflows
- Confident and right: $\ell \approx 0$; confident and **wrong**: ℓ grows like $|z|$

Why Not the Squared Error?



- For a confidently **wrong** prediction ($z \ll 0$) the cross-entropy gradient is -1: a strong push
- The squared error's gradient **vanishes** there: the model is stuck exactly when it is most wrong
- The cross-entropy is **convex** in (w, b); the squared error on a sigmoid is not

The Gradient Is a Residual Again

$$\nabla_w \mathcal{L} = \frac{1}{N} X^\top (p - y), \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_i (p_i - y_i)$$

Worked example: one e-mail, feature $x = 1.5$, weights $w = 1.2, b = -0.5$

	$y = 1$ (spam)	$y = 0$ (ham)
score $z = 1.2 \cdot 1.5 - 0.5$	1.300	1.300
probability $p = \sigma(z)$	0.786	0.786
loss $-\log p_y$	**0.241**	**1.541**
$\partial \ell / \partial w = (p - y)\, x$	-0.321	$+1.179$
$\partial \ell / \partial b = p - y$	-0.214	$+0.786$

- The canonical-link result of the GLM section, for the Bernoulli: (prediction — target) $\times$ feature. Not typed in the lab: `jax.grad` produces it

Training It: Convex but No Closed Form

- $\nabla \mathcal{L} = 0$ is **nonlinear** in w: no closed form. The loss is convex: GD, Adam or L-BFGS reach the global minimum
- Newton's method on it is IRLS (iteratively reweighted least squares)
- **Separable data:** the loss keeps decreasing as $\|w\| \to \infty$. **Regularization fixes it**

$$\mathcal{L}_\lambda = \frac{1}{N} \sum_i \ell_i + \frac{\lambda}{2} \|w\|^2, \qquad C = \frac{1}{\lambda N}$$

- Spam ($\lambda = 10^{-3}$): our weights equal scikit-learn's to $2 \cdot 10^{-6}$

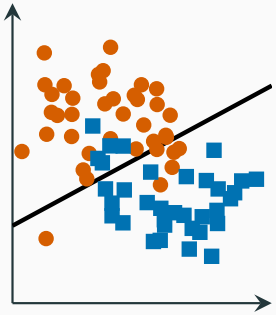
Result: Spam Detection

Model	Accuracy	Precision	Recall	F_1	MCC
majority class ("all ham")	0.606	0	0	0	0
perceptron, 1 epoch	0.912	0.875	0.906	0.890	0.817
logistic regression ($C = 0.1$)	**0.937**	**0.913**	**0.928**	**0.921**	**0.869**

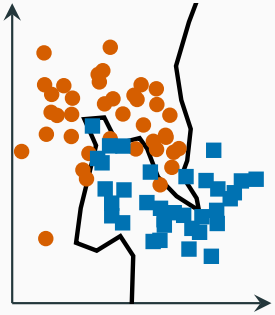
- Test set: 921 e-mails (363 spam). Log frequencies, standardized with the training statistics
- The logistic model is better than a single pass of the perceptron, *and* it outputs a probability per e-mail
- 92.8% of the spam is caught; **8.7%** of the e-mails flagged as spam are actually ham: the price of the default threshold of 0.5
- Next: what the numbers mean, and how to choose the threshold

Non-Linear Boundaries: Features and Regularization

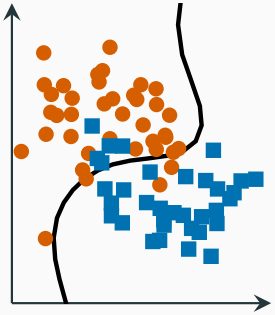
degree 1
train 0.84, test 0.848



degree 15, $C = 10^6$
train 0.955, test 0.859



degree 15, $C = 0.1$
train 0.860, test 0.876



- "Two moons", 200 noisy training points (a sample is drawn), 2000 test points; polynomial features + logistic regression
- Unregularized degree 15: an intricate boundary that fits the noise (train 0.955, test 0.859); with $C = 0.1$ it stays smooth and **generalizes better** (test 0.876)
- The best unregularized model here is degree 3 (test 0.887): flexibility and constraint both need tuning

More Than Two Classes: One Class Against the Rest

- A logistic regression is **binary**. For K classes, train K binary models: "class k **against all the others**"
- Predict $\hat{k} = \arg\max_k \; p_k(x)$ (separate models: the p_k need not sum to 1)
- scikit-learn: `OneVsRestClassifier(LogisticRegression())`; the same wrapper works for the perceptron
- Alternative: the **multinomial** logit (softmax) is the GLM for a categorical y: one weight vector per class, fitted *together*, probabilities sum to 1 (neural-network outputs, Class 05)

Live Demo: Logistic Regression

DEMO L1:

`03_logistic_regression.ipynb`

Classification Metrics

The Confusion Matrix

actual \ predicted	spam	ham
spam	**TP** = 337 spam caught	**FN** = 26 spam missed
ham	**FP** = 32 ham hidden	**TN** = 526 ham delivered

921 test e-mails
363 spam, 558 ham
(positive class: spam)

- Every metric of this section is a function of these **four counts** (the logistic model at threshold 0.5)
- The two errors are **not** equally bad: a false positive hides a real e-mail, a false negative shows one spam
- Multi-class: an $K \times K$ matrix; the diagonal is correct

Accuracy, Precision and Recall

$$\text{accuracy} = \frac{TP + TN}{n} = \frac{337 + 526}{921} = 0.937$$

$$\text{precision} = \frac{TP}{TP + FP} = \frac{337}{369} = 0.913$$

$$\text{recall} = \frac{TP}{TP + FN} = \frac{337}{363} = 0.928$$

- **Accuracy:** the fraction of correct decisions; misleading when the classes are imbalanced
- **Precision:** of the e-mails flagged as spam, how many really are? (cost of a false positive)
- **Recall** (sensitivity, true-positive rate): of the spam, how much was caught? (cost of a false negative)

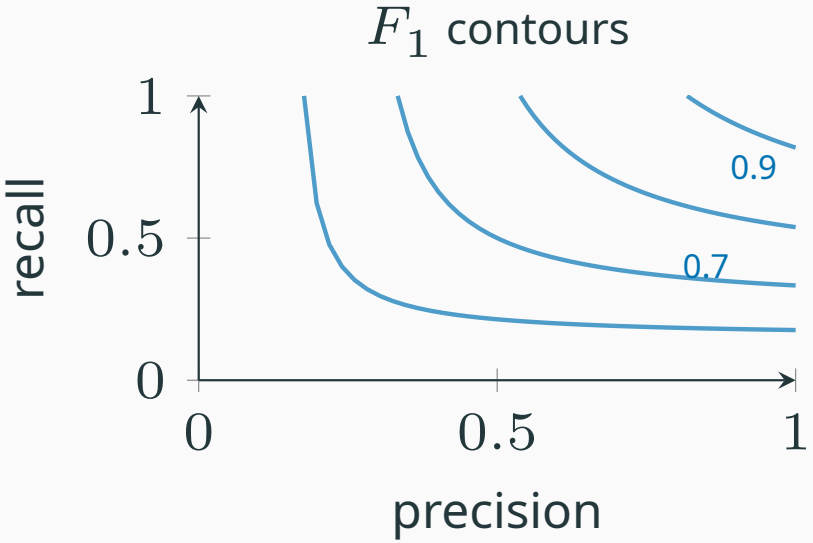
The Accuracy Paradox

Test set: all 558 ham + 11 spam (1.9% spam)	Accuracy	Precision	Recall	F_1	MCC
"everything is ham"	**0.981**	0	0	0	0
our classifier	0.942	0.238	0.909	0.377	0.449

- The useless filter has the **higher accuracy**: it never fails on the 98% that are ham
- Our classifier finds **10 of 11** spam, at the price of 32 false alarms (precision 0.238)
- Class 01: "the metric must match the goal". With rare positives, look at precision, recall and MCC, never accuracy alone
- Also: always compare with the **majority-class baseline** (0.606 on the balanced-ish spam data)

F_1: One Number for Precision and Recall

$$F_1 = \frac{2PR}{P+R} = \frac{2\,TP}{2\,TP + FP + FN} = \frac{674}{732} = 0.921$$



- The **harmonic mean**: low if *either* is low (precision 1, recall 0.01 gives $F_1 = 0.02$, not 0.5)
- Ignores the **true negatives**: F_1 can look fine for a model that is useless on the negative class

MCC: the Whole Matrix in One Number

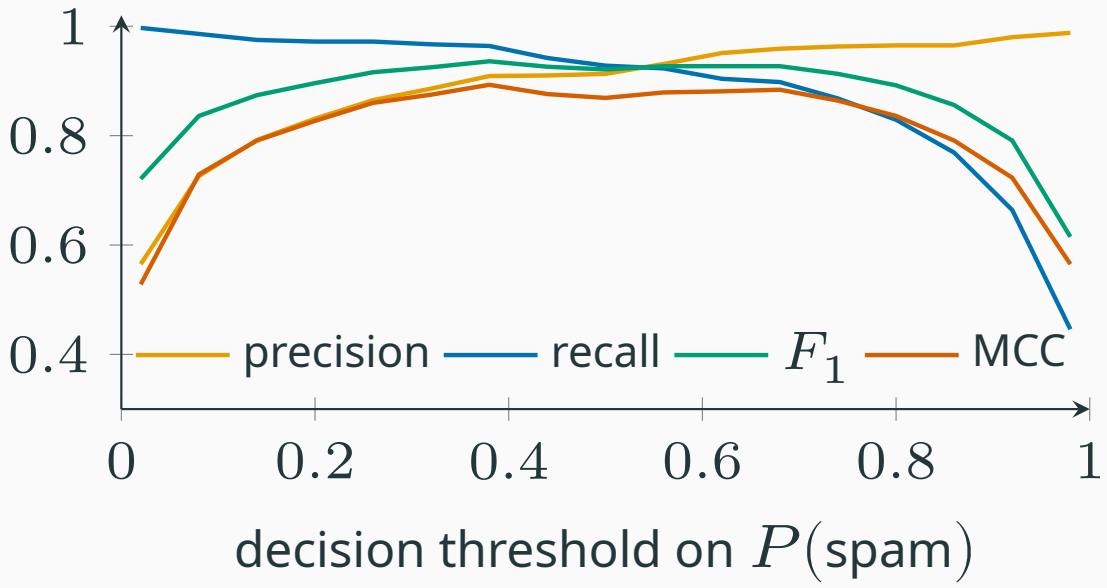
$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

$$\text{Ours: } \frac{337 \cdot 526 - 32 \cdot 26}{\sqrt{369 \cdot 363 \cdot 558 \cdot 552}} = \frac{176\,430}{203\,120} = 0.869$$

- The correlation of predicted and actual labels: $+1$ perfect, 0 chance, -1 total disagreement
- Uses all four cells: **symmetric** in the classes, robust to imbalance

"Everything is spam" (test set, 39.4% spam)	Value
accuracy	0.394
precision / recall	0.394 / 1.000
F_1	**0.565**
MCC	**0.000**

The Threshold Is a Decision



- Raising the threshold: fewer e-mails flagged, **precision up, recall down**. Lowering it: the reverse
- F_1 and MCC peak around 0.4–0.7; the best threshold for *our costs* may be far from it

Choosing the Threshold

Threshold	Precision	Recall	F_1	MCC	Ham hidden
0.50	0.913	0.928	0.921	0.869	32
0.90	0.973	0.705	0.818	0.749	7
0.99	0.993	0.386	0.556	0.521	**1**

- If hiding a real e-mail costs, say, 20 times a missed spam, the third row is the sensible one, at the price of **61% of the spam** getting through
- **Choose the threshold on validation data** with the costs in hand; the test set only reports the result (in the lab: precision 0.99 on the *training* data gives 0.974 on test)
- Threshold-free summaries: the **ROC curve** (recall against false-positive rate) and the **precision-recall curve**; AUC of ours: **0.981**
- Prefer PR curves when positives are rare

Which Metric When?

Situation	Look at	Why
balanced classes, equal costs	accuracy, MCC	simple, all cells
rare positives	precision, recall, PR curve	accuracy is inflated by negatives
a missed positive is costly (screening)	**recall**	do not miss sick patients
a false alarm is costly (spam filter)	**precision**	do not hide real e-mails
one number, any imbalance	**MCC**	uses TP, TN, FP, FN
ranking quality, threshold not fixed	ROC AUC, average precision	independent of the threshold

- Regression counterpart: MSE/RMSE for quadratic costs, MAE for robust, sMAPE for relative, R^2 against the mean
- Whatever the metric: baseline, spread over folds, and a **test set used once**

DEMO C1:

`04_classification_metrics.ipynb`

Summary

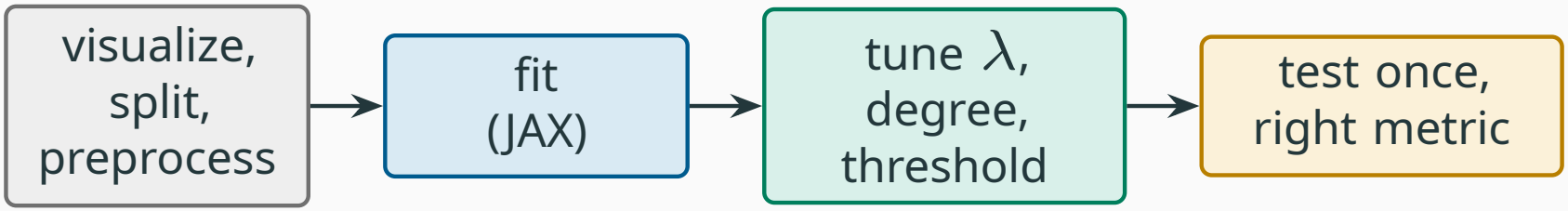
The Models Side by Side

	Linear regression	Perceptron	Logistic regression
Output	a number	a class	a probability
Loss	squared error	$\max(0, -m)$	cross-entropy
Training	closed form or GD	online updates	GD (convex)
Boundary	(hyperplane fit)	linear	linear
Assumes	linear mean, Gaussian noise	separability	linear log-odds
Result	$R^2 = 0.681$ (housing)	accuracy 0.912 (spam)	accuracy 0.937 (spam)

- Regularization (L_2, L_1) applies to every model that has weights; polynomial features make any of them non-linear
- Next class: where the losses come from (**maximum likelihood**) and a different kind of classifier (**Naive Bayes**)

Choosing and Checking

- **Start simple:** a baseline, then a linear model; add flexibility only if validation asks
- **A probability, not just a class:** logistic regression
- **Many features, few examples:** regularize; lasso to select features
- **Never trust a number without:** a baseline, a spread, a test set used once, the right metric
- **Watch for:** unscaled features, leakage, imbalance, a threshold left at 0.5



Key Takeaways

- **A model is (hypothesis, loss, optimizer):** write the loss in `jax.numpy`, `jax.grad` does the rest
- **Linear regression:** MSE, closed form or GD; **polynomial features** add flexibility (and overfitting)
- **Regularization:** ridge shrinks, lasso selects; λ is chosen on validation data
- **GLM:** linear predictor + link + distribution; the canonical-link gradient is always *features* $\times$ *residual*. **Logistic regression** is the Bernoulli GLM (cross-entropy); the **perceptron** is a subgradient step, not a GLM
- **Metrics:** MSE, MAE, sMAPE, R^2; confusion matrix, precision, recall, F_1, **MCC**; the threshold is a decision
- **Methodology:** visualize, split once, preprocess on training only, tune by CV, test once, always a baseline

Next Class: Probabilistic Models

- **Where do the losses come from?** Maximum likelihood: MSE is Gaussian noise, cross-entropy is Bernoulli labels
- **MAP:** a prior on the parameters; ridge is a Gaussian prior, Laplace smoothing a Beta prior
- **Naive Bayes:** Bayes' rule and independence; discrete and continuous features, for classes **and** numbers, trained by counting
- **Probabilities you can trust:** calibration, log-loss, and decisions with costs

References (1/2)

- G. James, D. Witten, T. Hastie, R. Tibshirani, J. Taylor, *An Introduction to Statistical Learning with Applications in Python*, 2023: chapters 3 (linear regression), 4 (classification), 6 (regularization)
- C. Bishop, *Pattern Recognition and Machine Learning*, 2006: chapters 3 and 4 (linear models)
- P. McCullagh, J. Nelder, *Generalized Linear Models*, 2nd ed., 1989

References (2/2)

- R. Tibshirani, “Regression shrinkage and selection via the lasso”, *JRSS B*, 1996
- H. Zou, T. Hastie, “Regularization and variable selection via the elastic net”, *JRSS B*, 2005
- D. Chicco, G. Jurman, “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation”, *BMC Genomics*, 2020
- F. Rosenblatt, “The perceptron: a probabilistic model for information storage and organization in the brain”, *Psychological Review*, 1958
- M. Minsky, S. Papert, *Perceptrons*, 1969